University of Mumbai

PROJECT DOCUMENTATION

SUBMITTED BY

SUNNY AMARLAL JAISWAR

SEAT NO: 2900010

Easy Go Web Project

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR QUALIFYING
MASTER OF SCIENCE IN
INFORMATION TECHNOLOGY
PART II (SEMESTER-IV)
EXAMINATION 2023-2024

DEPARTMENT OF INFORMATION TECHNOLOGY 3RD FLOOR, DR. SHANKAR DAYALSHARMA BHAVAN,
VIDYANAGRI, SANTACRUZ(EAST),
MUMBAI – 400098.

Easy Go Web Project

A Project Report

Submitted in partial fulfillment of the Requirements for the award of the Degree of

MASTER OF SCIENCE (INFORMATION TECHNOLOGY)

SUBMITTED BY

Sunny Amarlal

Jaiswar

Seat No.2900010

Under the esteemed guidance of

MR. BHUPINDER SINGH

PCP Center

Navneet College of Arts. Science & Commerce

Institute of Open and Distance Learning

(IDOL)

DR. SHANKAR DAYAL SHARMA BHAVAN, IDOL BUILDING,

VIDYANAGRI, SANTACRUZ (E), MUMBAI –400098

Academic Year: 2023-24

University of Mumbai

Institute of Distance and Open Learning (IDOL)

CERTIFICATE

This is to certify that Mr. **Sunny Amarlal Jaiswar** **Seat No. 2900010** of semester **IV** has completed the specified term on project titled **Easy Go Web Project** in satisfactorily manner within this institute as laid down by University of Mumbai during the academic year of 2023–24.

.

Incharge Coordinator

Internal Examiner External Examiner

Date College Seal

ABSTRACT

Easy Go Home is a web-based ridesharing platform designed to facilitate efficient and cost-effective transportation solutions. The platform connects riders with travelers sharing similar routes, allowing riders to monetize their journeys by offering rides to travelers. By leveraging technology to optimize vehicle occupancy and reduce transportation costs, Easy Go Home aims to promote sustainability, enhance mobility, and foster community engagement in the transportation sector.

ACKNOWLEDGEMENT

I express deep sense of gratitude to our respected and lamed guides, Prof. Bhupinder Singh (IT) of Navneet College for the guidance, we are thankful to them forthe encouragement and support they have given us in completing the project.

We are also thankful to all the other faculty and staff members of our department for their kind cooperation and help.

Lately, we would like to express our deep apperception towards our classmates and our parents for providing us the moral support and encouragement

DECLARATION

I hereby declare that the project entitled, “ ” done at NAVNEET COLLEGE OF ARTS, SCIENCE & COMMERCE, has not been in any case duplicated to submit to any other university for the award of any degree. To the best of my knowledge other than me, no one has submitted to any other university.The project is done in partial fulfillment of the requirements for the award of degree of **MASTERS OF SCIENCE (INFORMATION TECHNOLOGY)** to be submitted as final semester project as part of our curriculum.

<Name of Candidates>

CRTIFICATE

I am SUNNY AMARLAL JAISWAR. MSc.IT wishes to state that the work embodied in this project titled “EASY GO WEB PROJECT” forms my own contribution to the project work carried out under the guidance of Prof. at the department of Information Technology. This work has not been submitted for any other degree of this or any other university. Wherever references have been made to previous work of others, it has been clearly indicated assuch and included in the bibliography.

M.sc (IT) Co-Ordinator, IDOL External Examiner

INDEX

S.N	CONTENTS
1	TELESERV SYSTEM CERTIFICERTE
2	ACKNOWLEDGEMENT
3	DECLARATION
4	ABSTRACT
5	**CHAPTER 1** INTRODUCTION
6	ANALYSIS
7 7.1	SYSTEM ANALYSIS SYSTEM SPECIFICATIONS
8	**CHAPTER 2**
	DESIGN APPROACH
8.1	INTRODUCTION TO DESIGN
8.2	UML DIAGRAMS
8.3	DATA FLOW DIGRAMS
8.4	E-R DIAGRAMS
9	PROJECT MODULES
10	**CHAPTER 3**
	IMPLEMENTATION
10.1	CONCEPTS AND TECHNIQUES
10.2	TESTING
10.3	TEST CASES
11	**CHAPTER 4**
12	OUTPUTS SCREENS
13	CONCLUSION
14	BIBLIOGRAPHY

CHAPTE 1

INTRODUCTIO:

Welcome to Easy Go Home - your go-to platform for hassle-free ridesharing! Whether you're a rider with an empty seat or a traveler in need of a lift, Easy Go Home connects you effortlessly. Our user-friendly website simplifies the process, allowing riders to easily share their journey details and travelers to find suitable rides.

At Easy Go Home, we understand the importance of convenience and affordability when it comes to transportation. With just a few clicks, riders can list their upcoming trips, specifying the time, destination, and vehicle details. Travelers can then browse through available rides, select their preferred option, and complete a simple form to secure their seat. Our goal is to provide a seamless experience for both riders and travelers, fostering a community of sharing and reducing the environmental impact of individual travel. Join Easy Go Home today and embark on your next journey with ease!

ANALYSIS

SYSTEM ANALYSIS:

1. Existing System:

Currently, people rely on booking services like Ola or using public transport to get around. However, these options can be costly and inconvenient. With Easy Go Home, anyone can earn money by sharing their ride with others. It's a simple way to make extra cash while helping others get to their destinations efficiently and affordably.

2. Purpose System:

The purpose of Easy Go Home is to incentivize riders to offer rides to travelers during their solo journeys. By sharing their vehicle space with travelers, riders have the opportunity to earn additional income while contributing to a more sustainable transportation ecosystem. Easy Go Home encourages collaboration and resource optimization, empowering individuals to make efficient use of their vehicles and reduce their overall travel expenses.

3. Objective of the System:

The primary objective of Easy Go Home is to create a sustainable and cost-effective transportation solution by promoting ride sharing. By connecting riders with travelers who share similar routes, the system aims to optimize vehicle occupancy, reduce fuel consumption, and alleviate traffic congestion. Additionally, Easy Go Home seeks to enhance accessibility to transportation services and foster a sense of community among users.

System Specifications

Hardware Requirements:-

- Pentium-IV(Processor)
- 8 GB RAM
- Hard Disk 512 GB

Software Requirements:-

- **Operating System:**
- MAC / Linux / Windows

- **Backend:**
- SQL Server 2008,
- Core Java
- Advanced Java

- **Frontend:**
- HTML
- CSS
- JavaScript

- **Database:**
- MYSQL

CHAPTER 2

DESIGN APPROACH

INTRODUCTION:

Design is the first step in the development phase for any techniques and principles for the purpose of defining a device, a process or system in sufficient detail to permit its physical realization. Once the software requirements have been analyzed and specified the software design involves three technical activities - design, coding, implementation, and testing that are required to build and verify the software.

Design activities are of main importance in this phase, because in this activity, decisions ultimately affecting the success of the software implementation and its ease of maintenance are made. These decisions have the final bearing upon reliability and maintainability of the system. Design is the only way to accurately translate the customer's requirements into finished software or a system. Design is the place where quality is fostered in development. Software design is a process through which requirements are translated into a representation of software. Software design is conducted in two steps. Preliminary design is concerned with the transformation of requirements into data**.**

UML Diagrams:

Actor:

A coherent set of roles that users of use cases play when interacting with the use `cases.

Use case:

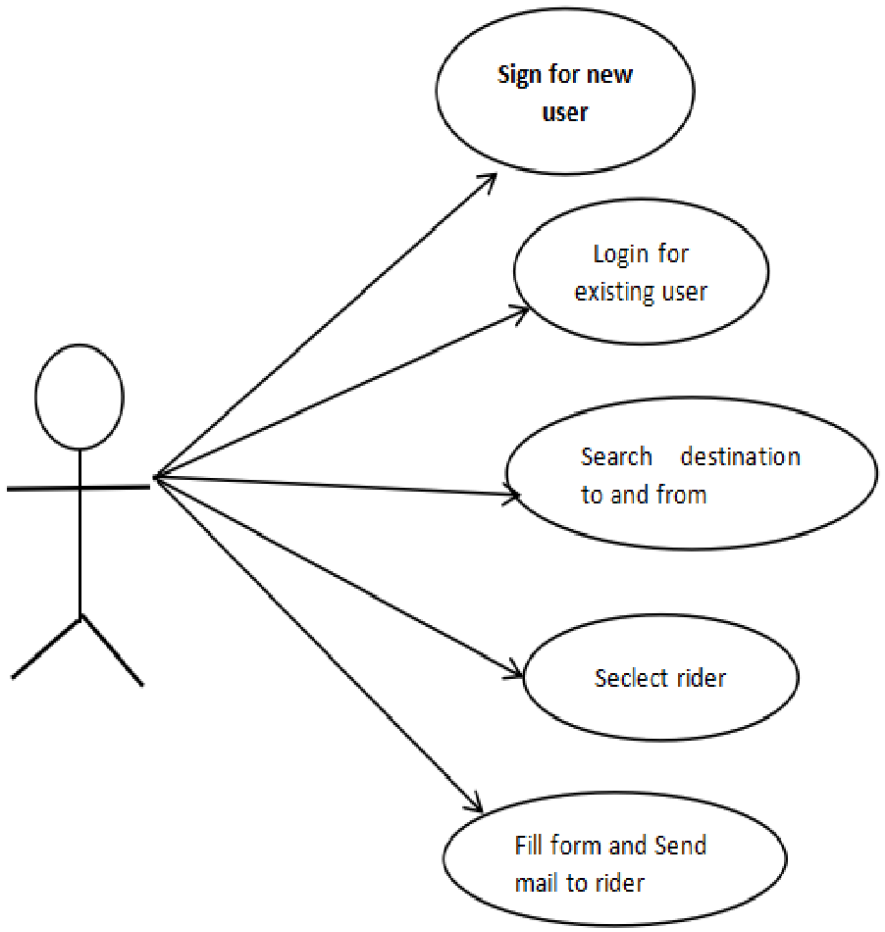
A description of sequence of actions, including variants, that a system performs that yields an observable result of value of an actor. UML stands for Unified Modeling Language. UML is a language for specifying, visualizing and documenting the system. This is the step while developing any product after analysis. The goal from this is to produce a model of the entities involved in the project which later need to be built. The representation of the entities that are to be used in the product being developed need to be designed.

There are various kinds of methods in software design:

They are as follows:
Use case Diagram
COLLABORATION DIAGRAM:
State Chart Diagram

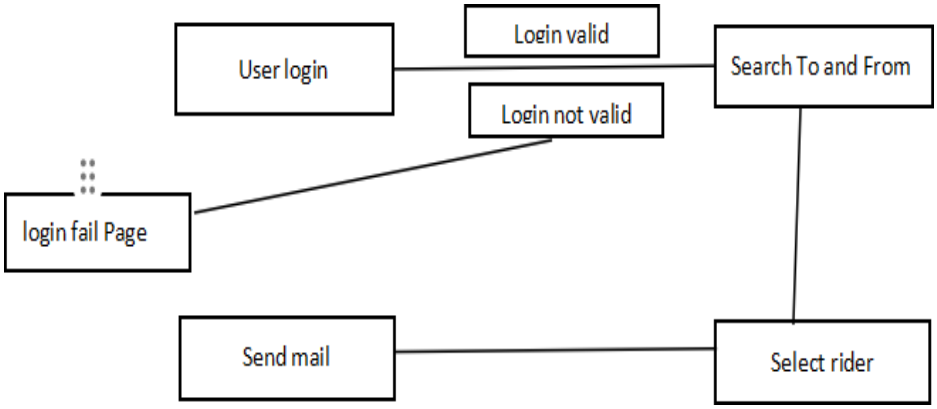
USECASE DIAGRAMS:

Use case diagrams model behavior within a system and help the developers understand what the user require. The stick man represents what's called an actor. Use case diagram can be useful for getting an overall view of the system and clarifying who can do and more importantly what they can't do. A use case diagram consists of use cases and actors and shows the interaction between the use case and actors. The purpose is to show the interactions between the use case and actor. To represent the system requirements from user's perspective. An actor could be the end-user of the system or an external system.

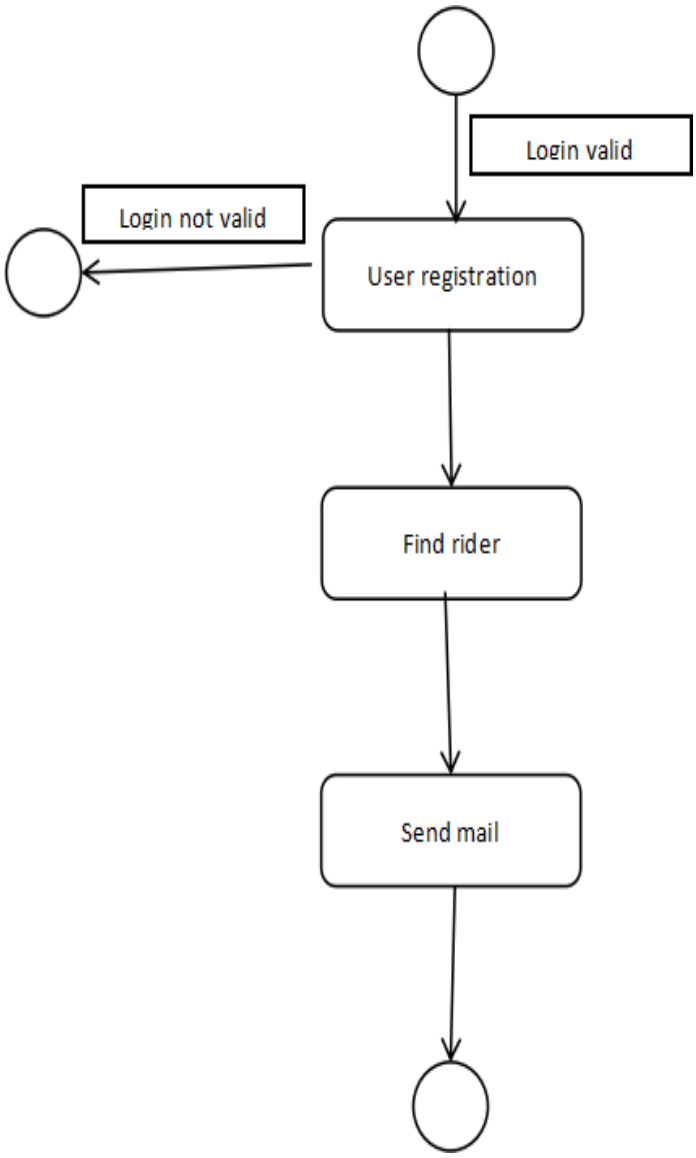

Sign for new user
Login for existing user
Search destination to and from
Seclect rider
Fill form and Send mail to rider

COLLABORATION DIAGRAM:

A collaboration diagram is an introduction diagram that emphasizes the structural organization of the objects that send and receive messages. Graphically a collaboration diagram is a collection of vertices and arcs.



STATE CHART DIAGRAM:



DATA FLOW DIAGRAMS

The DFD takes an input-process-output view of a system i.e. data objects flow into the software, are transformed by processing elements, and resultant data objects flow out of the software. Data objects represented by labeled arrows and transformation are represented by circles also called bubbles presented in a hierarchical fashion. the first data flow model represents the system as a whole. Subsequent DFD refine the context diagram (level 0 DFD), providing increasing details with each subsequent level.

The DFD enables the software engineer to develop models of the information domain & functional domain at the same time. As the DFD is refined into greater levels of detail, the analyst performs an implicit functional decomposition of the system. At the same time, the DFD refinement results in a corresponding refinement of the data as it moves through the process that embody the applications. A context-level DFD for the system the primary external entities produce information for use by the system and consume information generated by the system. The labeled arrow represents data objects or object hierarchy

RULES FOR DFD:

Fix the scope of the system by means of context diagrams. Organize the DFD so that the main sequence of the actions Reads left to right and top to bottom. Identify all inputs and outputs. Identify and label each process internal to the system with Rounded circles. A process is required for all the data transformation and Transfers. Therefore, never connect a data store to a data Source or the destinations or another data store with just a Data flow arrow. Do not indicate hardware and ignore control information. Make sure the names of the processes accurately convey everything the process is done. There must not be an unnamed process. Indicate external sources and destinations of the data, with Squares. Number each occurrence of repeated external entities. Identify all data flows for each process step, except simple Record retrievals. Label data flow on each arrow. Use details flow on each arrow. Use the details flow arrow to indicate data movements.

E-R Diagrams:

The Entity-Relationship (ER) model was originally proposed by Peter in 1976 [Chen76] as a way to unify the network and relational database views. Simply stated the ER model is a conceptual data model that views the real world as entities and relationships. A basic component of the model is the Entity-Relationship diagram which is used to visually represent data objects. Since Chen wrote his paper the model has been extended and today it is commonly used for database design for the database designer, the utility of the ER model is:

Connectivity and Cardinality

The basic types of connectivity for relations are one-to-one, one-to-many, and many-to many. A *one-to-one* (1:1) relationship is when at most one instance of a entity A is associated with one instance of entity B. For example, "employees in the company are each assigned their own office. For each employee there exists a unique office and for each office there exists a unique employee. A *one-to-many* (1: N) relationship is when for one instance of entity A, there are zero, one, or many instances of entity B, but for one instance of entity B, there is only one instance of entity A. An example of a 1: N relationships is a department has many employees each employee is assigned to one department A *many-to-many* (M: N) relationship, sometimes called non-specific, is when for one instance of entity A, there are zero, one, or many instances of entity B and for one instance of entity B there 20 are zero, one, or many instances of entity A. The connectivity of a relationship describes the mapping of associated.

ER Notation

There is no standard for representing data objects in ER diagrams. Each modeling methodology uses its own notation. The original notation used by Chen is widely used in academics texts and journals but rarely seen in either CASE tools or publications by non- academics. Today, there are several

notations used, among the more common are Bachman, crow's foot, and IDEFIX. All notational styles represent entities as rectangular boxes and relationships as lines connecting boxes. Each style uses a special set of symbols to represent the cardinality of a connection. The notation used in this document is from Martin. The symbols used for the basic ER constructs are:

Entities are represented by labeled rectangles. The label is the name of the entity. Entity names should be singular nouns.

relationships are represented by a solid line connecting two entities. The name of the relationship is written above the line. Relationship names should be verbs

attributes, when included, are listed inside the entity rectangle. Attributes which are identifiers are underlined. Attribute names should be singular nouns.

cardinality of many is represented by a line ending in a crow's foot. If the crow's foot is omitted, the cardinality is one.

existence is represented by placing a circle or a perpendicular bar on the line. Mandatory existence is shown by the bar (looks like a 1) next to the entity for an instance is required. Optional existence is shown by placing a circle next to the entity that is optional 21

PROJECT MODULES

MODULES:

Ride Management:
Listing Rides
Searching Rides
Booking Rides
Cancelling Rides

User Profile:
Registration
Login
Profile Editing
Password Reset

Payment Handling:
Payment Processing
Payment Verification
Payment Refunds
Transaction History

Notifications:
Ride Requests
Ride Confirmations
Payment Notifications
Account Notifications

MODULE DESCRIPTIOIN:

1) **User Management:**

This module handles user registration, login, and profile management functionalities. Users can create accounts, log in securely, and update their profile information as needed.

2) **Ride Management:**

The Ride Management module facilitates the listing, search, and booking of rides. Riders can list their upcoming trips, while travelers can search for available rides based on their destination and schedule. Booking functionalities allow travelers to reserve seats for their desired rides.

3) Payment Gateway:

This module integrates a secure payment gateway to facilitate transactions between riders and travelers. It enables travelers to make payments for ride bookings securely, while riders receive payments for offering rides to travelers.

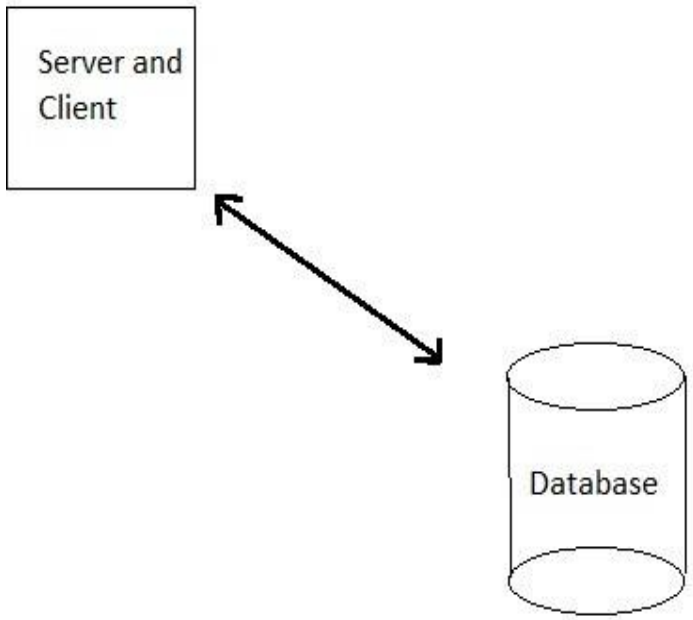
4) Notification System:

The Notification System module sends out automated email notifications to users regarding ride requests, confirmations, updates, and reminders. It ensures timely communication between riders and travelers, enhancing the overall user experience.

DATABASE MODELS

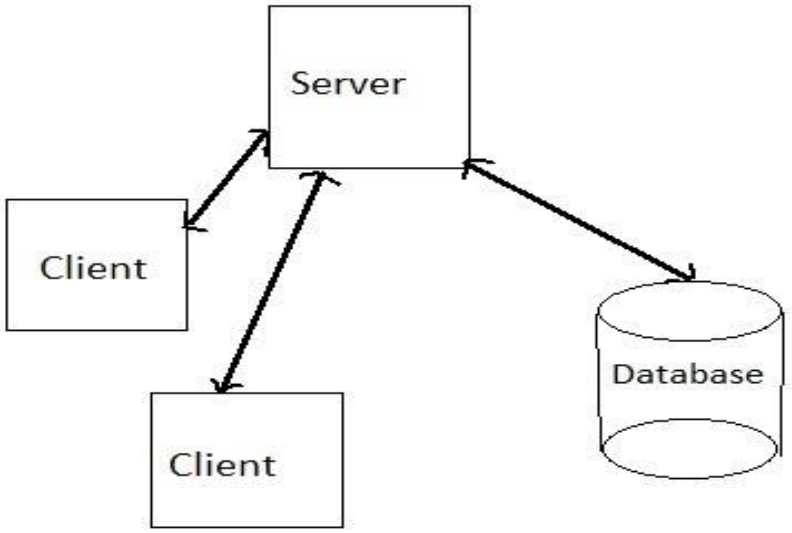
The Easy Go Home database models consist of several interconnected entities designed to efficiently manage and store data related to ridesharing activities. At the core of the database is the "User" entity, which stores information about registered users, including their personal details and login credentials. Each user can interact with the system by either offering rides as a "Rider" or booking rides as a "Traveler." These roles are represented by separate entities, with relationships established to track the association between users and their respective roles.

Additionally, the database includes entities such as "Ride," "Booking," and "Payment," which capture essential details about rides offered, ride bookings made by travelers, and payment transactions processed within the system. Relationships between these entities ensure that relevant data is appropriately linked, enabling seamless retrieval and manipulation of ride-related information. Through well-defined database models, Easy Go Home ensures efficient data management, facilitating smooth operations and enhancing the overall user experience for both riders and travelers.



Two Tier (client-server)

In two tier architecture the database resides in one machine and the client in different machine they are connected through the network. In this type of architecture, database management takes control of the database and provides access to clients in a network. This software bundle is also called the server. Software in different machines, requesting for information are called as the clients.



Three Tier and N-Tier

In the three-tier architecture, any number of servers can access the database that resides on server. Which in turn serves clients in a network. For example, you want to access the database using java applets, the applet running in some other machine, can send request only to the server from which it is downloaded. For this reason we will need to have a intermediate server which will accept the requests from applets and then to the actual database server. This intermediate server acts as a two-way communication channel also. This is the information or data from the database that is passed on to the applet that is requesting it. This can be extended to make n tiers of servers, each server carrying to specific type of request from clients, however in practice only 3 tiers architecture is popular.

Java Language:

History: Java was developed by James Gosling and his team at Sun Microsystems in the early 1990s. It was originally designed for interactive television, but its versatility and portability made it an ideal choice for web development, enterprise applications, and mobile development. Java has since become one of the most popular programming languages globally.

Features:

Object-Oriented: Java is a purely object-oriented programming language, which means it focuses on creating reusable and modular code through objects and classes.
Platform Independence: Java programs can run on any platform that supports Java Virtual Machine (JVM), making it platform independent. Robustness: Java has features like automatic memory management (garbage collection), exception handling, and strong type checking, which contribute to its robustness. Security: Java has built-in security features such as sandboxing, which protects against malicious code execution. Multi-threading: Java supports multi-threading, allowing concurrent execution of multiple tasks within a program.

Data Types:

Java has two categories of data types:
Primitive Data Types: byte, short, int, long, float, double, Boolean, char. Reference Data Types: objects, arrays, interfaces, Enums. Servlet: Servlets are Java classes that extend the capabilities of servers. They are used to create dynamic web content and handle HTTP requests and responses. Servlets are typically used in web development to generate HTML pages dynamically, interact with databases, and perform other server-side tasks.

JDBC (Java Database Connectivity):
JDBC is a Java API that allows Java applications to interact with relational databases. It provides methods for connecting to databases, executing SQL queries, and retrieving results. JDBC acts as a bridge between Java applications and databases, enabling seamless data access and manipulation.

JSP (Java Server Pages):

JSP is a technology used to create dynamic web pages in Java. It allows developers to embed Java code within HTML pages, making it easier to generate dynamic content. JSP pages are compiled into servlets by the web container at runtime, enabling the execution of Java code to generate dynamic content on the server-side. JSP is commonly used in web development to create dynamic and interactive web applications.

DATA BASE TABLES

Login Table

id	username	password	email
1	sunny	5555	abc@gamil.com
2	vijay	555	abc@gamil.com
3	sunny	8888	abcdfk@gamil.com
4	dheeraj	7777	abcccc@gamil.com

Rider Table and traveler data

userName	userPhone	current Date	fromTime	toTime	userEmail	userGender	fromStation	toStation	typeOfWheeler
Rudra	9185513155	2023-09-20	22:38	23:38	rudra@gmail.com	male	Vasai	Miraroad	four wheeler
shiva	9788888999	2023-09-20	18:18	19:18	shiva@gmail.com	male	Vasai	Miraroad	four wheeler
sanky	9755555555	2023-09-19	21:25	22:25	jhb@gmail.com	male	Vasai	Bhayandar	four wheeler
vicky	9772222277	2023-09-20	8:18	9:19	viky@gmail.com	male	Vasai	Dahisar	
santosh	9755555666	2023-09-13	15:55	16:55	santosh@gmail.com	male	Vasai	Dahisar	four wheeler
aniiiii	9788888977	2023-09-20	14:57	15:57	anii@gmail.com	male	Vasai	Dahisar	four wheeler

Technical Feasibility:

The Easy Go Home project is technically feasible as it leverages widely-used technologies like Java, Adv. Java, and SQL Server, ensuring compatibility and scalability across different platforms and environments.

Economical Feasibility:

Easy Go Home demonstrates economic feasibility by offering a cost-effective solution for ride sharing, allowing users to save on transportation expenses while providing an opportunity for riders to earn extra income through shared rides.

Operational Feasibility:

Easy Go Home is operationally feasible as it simplifies the process of ridesharing through user-friendly interfaces and automated functionalities, streamlining operations for both riders and travelers.

CHAPTER 3

Implementation:

Chapter 3: Implementation

In the implementation phase of the Easy Go Home project, Java programming language is utilized extensively for both front-end and back-end development to ensure a robust and efficient system.

1) Front-End Development:

HTML, CSS, and JavaScript: Front-end interfaces are developed using HTML for structure, CSS for styling, and JavaScript for interactive elements. This allows for the creation of user-friendly and responsive web pages. JSP (Java Server Pages): Java Server Pages are used to dynamically generate HTML content, enabling the integration of Java code with HTML to render dynamic web pages.

2) Back-End Development:

Core Java: Core Java is employed for implementing server-side logic, including handling user requests, processing data, and managing user sessions. Servlets: Servlets play a crucial role in the back-end architecture, serving as the foundation for handling HTTP requests and generating dynamic responses. They facilitate communication between the client-side interfaces and the server-side logic. JDBC (Java Database Connectivity): JDBC is utilized for database connectivity, enabling Java applications to interact with the backend database (e.g., MySQL) to retrieve and manipulate data.

ASP .NET: ASP .NET is integrated into the implementation for specific functionalities, leveraging its features for web application development and enhancing the overall functionality of the system.By implementing the Easy Go Home project in Java, developers can ensure a scalable, maintainable, and secure ridesharing platform that meets the requirements of both riders and travelers, providing a seamless experience for all users.

TESTING

Purpose: Unit testing is conducted to verify the functionality of individual units or components of the Easy Go Home system, ensuring that each unit performs as expected.
Approach: Test cases are designed to validate the behavior of specific methods, classes, or modules, with inputs and expected outputs defined for each test scenario.
Tools: JUnit or TestNG frameworks are utilized to automate unit tests, allowing for the execution of test cases and assertion of results in a controlled environment.
Integration Testing:

Purpose: Integration testing verifies the interactions between different modules or components of the Easy Go Home system, ensuring seamless communication and functionality across integrated units.
Approach: Test scenarios are designed to simulate interactions between interconnected modules, validating data flow, error handling, and system interfaces.
Tools: Integration testing frameworks such as Mockito or Power Mock are employed to simulate dependencies and interactions between modules, facilitating comprehensive integration testing.
System Testing:

Purpose: System testing evaluates the overall behavior and functionality of the Easy Go Home system, ensuring that it meets specified requirements and user expectations.
Approach: Test scenarios cover end-to-end user workflows, including user registration, ride booking, payment processing, and notification handling, validating system functionality and user experience.
Tools: Selenium WebDriver is used for automated system testing, allowing for the execution of test scripts that simulate user interactions with the Easy Go Home web application.

User Acceptance Testing (UAT):

Purpose: User acceptance testing involves validating the Easy Go Home system against real-world user scenarios and requirements, ensuring that it meets user needs.
Approach: Test scenarios are designed based on user stories and feedback, allowing end users to interact with the system and provide feedback on usability, performance, and functionality.
Tools: Manual testing techniques, user feedback forms, and usability testing sessions are employed to gather user feedback and insights, guiding improvements, and refinements to the Easy Go Home.

Maintenance and environment:

Maintenance:

Purpose: Maintenance of the Easy Go Home system is essential for ensuring its continued functionality, reliability, and performance over time.

Activities: Maintenance tasks include bug fixing, performance optimization, security updates, and feature enhancements based on user feedback and evolving requirements.

Approach: A proactive maintenance strategy is adopted, which involves regular monitoring, proactive identification of issues, and timely resolution to minimize system downtime and ensure uninterrupted service.

Environment:

Purpose: The environment in which the Easy Go Home system operates plays a critical role in its performance, scalability, and security.

Infrastructure: The system is hosted on reliable and scalable cloud infrastructure providers such as AWS, Azure, or Google Cloud Platform, ensuring high availability and scalability.

Security Measures: Robust security measures, including firewalls, encryption, and intrusion detection systems, are implemented to protect against cyber threats and safeguard user data.

Performance Optimization: The environment is optimized for performance, with adequate resources allocated to handle expected user traffic and peak loads efficiently. Regular performance testing and optimization ensure optimal system performance and responsiveness.

CORRECTION

Purpose: Maintenance of the Easy Go Home system is essential to ensure its continued functionality, reliability, and performance over time.

Activities: Maintenance tasks include bug fixing, performance optimization, security updates, and feature enhancements based on user feedback and evolving requirements.

Approach: A proactive maintenance strategy is adopted, involving regular monitoring, proactive issue identification, and timely resolution to minimize system downtime and ensure uninterrupted service.

Environment:

Purpose: The environment in which the Easy Go Home system operates plays a critical role in its performance, scalability, and security.

Infrastructure: The system is hosted on reliable and scalable cloud infrastructure providers such as AWS, Azure, or Google Cloud Platform, ensuring high availability and scalability.

Security Measures: Robust security measures, including firewalls, encryption, and intrusion detection systems, are implemented to protect against cyber threats and safeguard user data.

Performance Optimization: The environment is optimized for performance, with adequate resources allocated to handle expected user traffic and peak loads efficiently. Regular performance testing and optimization ensure optimal system performance and responsiveness.

TESTING

Testing is a process of executing a program with the indent of finding an error. Testing is a crucial element of software quality assurance and presents ultimate review of specification, design and coding. System Testing is an important phase. Testing represents an interesting anomaly for the software. Thus a series of testing are performed for the proposed system before the system is ready for user acceptance testing. A good test case is one that has a high probability of finding an as undiscovered error. A successful test is one that uncovers an as undiscovered error

Testing Objectives:

1.Testing is the process of executing a program with the intent of finding an error.

2. A good test case is one that has a probability of finding a yet undiscovered error.

3.A successful test is one that uncovers an undiscovered error.

Testing Principles:

1. All tests should be traceable to end user requirements.
2. Tests should be planned long before testing begins.
3. Testing should begin on a small scale and progress towards testing on a large.
4. Exhaustive testing is not possible.
5. To be most effective testing should be conducted by a independent third party.
6. The primary objective for test case design is to derive a set of tests that has the highest livelihood for uncovering defects in software.
7. To accomplish this objective two different categories of test case design techniques are used.

They are

White box testing.
Black box testing.
White-box testing:

White box testing focuses on the program control structure. Test cases are derived to ensure that all statements in the program have been executed at least once during testing and that all logical conditions have been executed.

Block-box testing:

Black box testing is designed to validate functional requirements without regard to the internal workings of a program. Black box testing mainly focuses on the information. domain of the software, deriving test cases by partitioning input and output in a manner that provides through test coverage. Incorrect and missing functions, interface errors, errors in data structures, error in functional logic are the errors falling in this category.

Testing strategies:

A strategy for software testing must accommodate low-level tests that are necessary to verify that all small source code segment has been correctly implemented as well as high-level tests that validate major system functions against customer requirements.

Testing fundamentals:

Testing is the process of executing a program with the intent of finding error. A good test case is one that has a high probability of finding an undiscovered error. If testing is conducted successfully, it uncovers the errors in the software. Testing cannot show the absence of defects, it can only show that software defects are present.

Testing Information flow:

Information flow for testing flows the pattern. Two classes of input were provided to test the process. The software configuration includes a software requirements specification, a design specification and source code. 51 Test configuration includes test plan and test cases and test tools. Tests are conducted and all the results are evaluated. That is test results are compared with expected results. When erroneous data is uncovered, an error is implied and debugging commences.

Unit testing:

Unit testing is essential for the verification of the code produced during the coding phase and hence the goal is to test the internal logic of the modules. Using the detailed design description as a guide, important paths are tested to uncover errors within the boundary of the modules. These tests were carried out during the programming stage itself. All units of Vienna SQL were successfully tested.

Integration testing:

Integration testing focuses on unit tested modules and build the program structure
that is dictated by the design phase.

System testing:

System testing tests the integration of each module in the system. It also tests to find discrepancies between the system and it's original objective, current specification and system documentation. The primary concern is the compatibility of individual modules. The entire system is working properly or not will be tested here, and specified path ODBC connection will correct or not, and giving output or not are tested here these verifications and validations are done by giving input values to the system and

by comparing with expected output. Top-down testing implementing here.

Acceptance Testing:

This testing is done to verify the readiness of the system for the implementation. Acceptance testing begins when the system is complete. Its purpose is to provide the end user with confidence that the system is ready for use. It involves planning and 52 execution of functional tests, performance tests and stress tests in order to demonstrate that the implemented system satisfies its requirements. Tools to special importance during acceptance testing include Test coverage Analyzer – records the control paths followed for each test case. Timing Analyzer – also called a profiler, reports the time spent in various regions of the code are areas to concentrate on to improve system performance. Coding standards – static analyzers standard checkers are used to inspect code for deviations from standards and guidelines.

Test Cases:

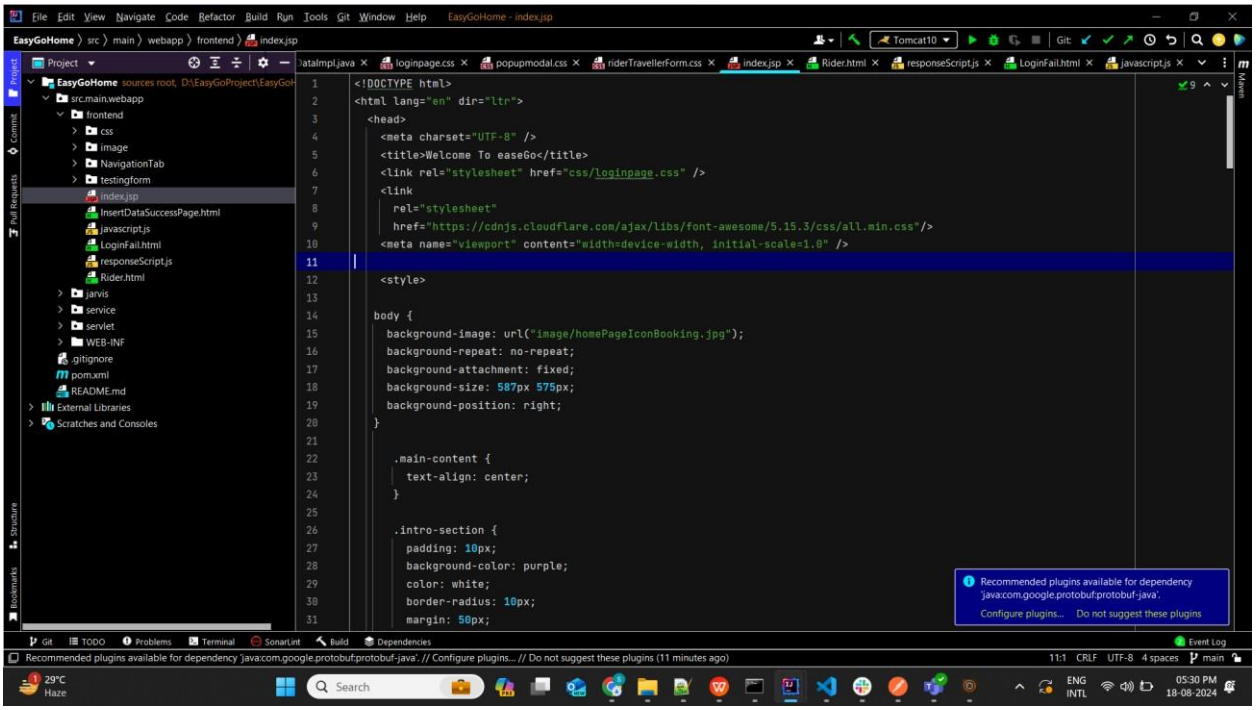
Test cases are derived to ensure that all statements in the program have been executed at least once during testing and that all logical conditions have been executed. Using White-Box testing methods, the software engineer can drive test cases that Guarantee that logical decisions on their true and false sides. Exercise all logical decisions on their true and false sides.

Execute all loops at their boundaries and within their operational bounds. Exercise internal data structure to assure their validity. The test case specification for system testing has to be submitted for review before system testing commences.

CHAPTER 4

FrontEnd Code :

HTML ,CSS, JAVASCRIPT



Package hierarchy: EasyGoHome→Src→Main→Webapp→FrontEnd

1. **HTML (index.html)**

```
<!DOCTYPE html>
<html lang="en" dir="ltr">
  <head>
    <meta charset="UTF-8" />
    <title>Welcome To easeGo</title>
    <link rel="stylesheet" href="css/loginpage.css" />
    <link
      rel="stylesheet"
      href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/5.15.3/css/all.min.css"/>
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />

    <style>

   body {
     background-image: url("image/homePageIconBooking.jpg");
     background-repeat: no-repeat;
     background-attachment: fixed;
     background-size: 587px 575px;
     background-position: right;
   }
```

```
.main-content {
  text-align: center;
}

.intro-section {
  padding: 10px;
  background-color: purple;
  color: white;
  border-radius: 10px;
  margin: 50px;
}

.intro-section h1 {
  font-size: 2.5em;
  margin-bottom: 10px;
}

.intro-section p {
  font-size: 1em;
}

.features-section {
  margin-top: 40px;
  display: flex;
  justify-content: center;
  flex-wrap: wrap;
}

.features-section h2 {
  width: 100%;
  font-size: 1.5em;
  margin-bottom: 2px;
}

.feature-item {
  background-color: #ffffff;
  padding: 0px;
  margin: 10px;
  border-radius: 10px;
  box-shadow: 0 3px 10px rgba(0, 0, 0, 1.1);
  width: 225px;
  transition: transform 0.3s ease;
}

.feature-item:hover {
  transform: scale(1.05);
}

.feature-icon {
  width: 80px;
```

```
      height: 80px;
      margin-bottom: 15px;
    }

    .feature-item h3 {
      font-size: 1.1em;
      margin-bottom: 10px;
    }

    .feature-item p {
      font-size: 1em;
    }
  </style>
</head>
<body>
  <div class="navigation">
    <nav>
      <div class="navbar">
        <div class="logo">
          <a href="http://localhost:8050/EasyGoHome/"><i class="fas fa-home"></i> EasyGo
Home</a>
        </div>
        <ul class="menu">
          <li>
            <a
href="http://localhost:8050/EasyGoHome/NavigationTab/CategoryPage.html">Category</a>
          </li>
          <li>
            <a
href="http://localhost:8050/EasyGoHome/NavigationTab/AboutUsPage.html">About</a>
          </li>
          <li>
            <a
href="http://localhost:8050/EasyGoHome/NavigationTab/ContactUsPage.html">Contact</a>
          </li>
        </ul>
        <button class="loginBtn" id="loginForm">Login</button>
      </div>
    </nav>
  </div>
  <div class="container login-signup_form dsp-none">
    <input type="checkbox" id="flip" />
    <div class="cover">
      <div class="front">
        <img src="image/goku.jpg" alt="" />
        <div class="text">
          <span class="text-1">Every new friend is a <br />
            new adventure</span>
          <span class="text-2">Let's get connected</span>
        </div>
      </div>
```

```
      <div class="back">
        <div class="text">
          <span class="text-1">Complete miles of journey <br />
            with one step</span>
          <span class="text-2">Let's get started</span>
        </div>
      </div>
    </div>
    <div class="forms">
      <button class="closeBtn" id="close">&times;</button>
      <div class="form-content">
        <div class="login-form">
          <div class="title">Login</div>
          <form method="post" action="userDetail">
            <div class="input-boxes">
              <div class="input-box">
                <i class="fas fa-envelope"></i>
                <input type="text" name="e1" placeholder="Enter your email" required/>
              </div>
              <div class="input-box">
                <i class="fas fa-lock"></i>
                <input type="password" name="p1" placeholder="Enter your password"  required
/>
              </div>
              <div class="text">
                <a href="http://localhost:8080/ServletTest/forget.jsp" >Forgot password?</a>
              </div>
              <div class="button input-box">
                <input type="submit" value="Sumbit" />
              </div>
              <div class="text sign-up-text">
                Don't have an account? <label for="flip">Sigup now</label>
              </div>
            </div>
          </form>
        </div>
        <div class="signup-form">
          <div class="title">Signup</div>
          <form method="get" action="userDetail">
            <div class="input-boxes">
              <div class="input-box">
                <i class="fas fa-user"></i>
                <input type="text" name="n1" placeholder="Enter your name" required />
              </div>
              <div class="input-box">
                <i class="fas fa-envelope"></i>
                <input type="text" name="e1" placeholder="Enter your email" required />
              </div>
              <div class="input-box">
                <i class="fas fa-lock"></i>
                <input type="password" name="p1" placeholder="Enter your password" required
```

```
/>
            </div>
            <div class="button input-box">
              <input type="submit" value="Sumbit" />
            </div>
            <div class="text sign-up-text">
              Already have an account? <label for="flip">Login now</label>
            </div>
          </div>
        </form>
      </div>
    </div>
  </div>

  <div class="main-content">
    <div class="intro-section">
      <h3>Welcome to EasyGo</h3>
      <p>
        ultimate Ride booking and Ride sharing website. Simplify your daily
        life journey.
      </p>
    </div>
    <div class="features-section">
      <h2>Features</h2>
      <div class="feature-item">
        <img src="image/HomeCabBookingIcone.jpg" alt="Easy Booking" class="feature-
icon"/>
        <h3>Easy Booking</h3>
        <p>Book your ride easily with our user-friendly interface.</p>
      </div>
      <div class="feature-item">
        <img src="image/HomeIconRealTimeTraking.jpg" alt="Real-Time Tracking"
class="feature-icon" />
        <h3>Real-Time Tracking</h3>
        <p>Track your ride in real-time with our accurate tracking system.</p>
      </div>
      <div class="feature-item">
        <img src="image/HomeSecurePay.jpg" alt="Secure Payments" class="feature-icon" />
        <h3>Secure Payments</h3>
        <p>Enjoy secure and hassle-free payment options.</p>
      </div>
    </div>
  </div>

  <footer>
    <p>&copy; 2023 EasyGo - All rights reserved.</p>
  </footer>
 </body>
 <script src="javascript.js"></script>
</html>
```

2. Rider.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0-
beta3/css/all.min.css">
    <link rel="stylesheet" href="css/riderTravellerForm.css">
    <link rel="stylesheet" href="css/popupmodal.css">
    <title>EasyGo</title>
</head>
<body style="background-image: url(image/RiderTravellerImage.jpg);
background-repeat: no-repeat;background-attachment: fixed;
background-size: 735px 675px; background-position: right;">
<header>
    <div class="logo">
        <a href="http://localhost:8050/EasyGoHome/"><i class="fas fa-home"></i> EasyGo
Home</a>
    </div>
    <h2>Start Journey With EasyGo </h2>
    <div class="topnav">
        <a href="javascript:void(0);" class="icon" onclick="toggleMenu()">
            <i class="fa fa-bars"></i>
        </a>
    </div>
</header>
<div id="myLinks" class="card dsp-none">
    <a class="fonthamburger"
href="http://localhost:8050/EasyGoHome/NavigationTab/CategoryPage.html">Category</a>
    <a class="fonthamburger"
href="http://localhost:8050/EasyGoHome/NavigationTab/AboutUsPage.html">About  Us</a>
    <a class="fonthamburger"
href="http://localhost:8050/EasyGoHome/NavigationTab/ContactUsPage.html">Contact
Us</a>
</div>
<section class="container formOne dsp-none">
    <form method="get" action="riderData" class="form">
        <h3 class="registrationForm">Driver Registration Form</h3></br></br>
        <div class="input-box">
            <label>Full Name</label>
            <input type="text" name="name" placeholder="Enter full name" required />
        </div>
        <div class="input-box">
            <label>Email Address</label>
            <input type="text" name="email" placeholder="Enter email address" required />
        </div>
```

```
<div class="gender-box">
    <h3>Gender</h3>
    <div class="gender-option">
        <div class="gender">
            <input type="radio" id="check-male" name="gender" value="male" />
            <label for="check-male">Male</label>
        </div>
        <div class="gender">
            <input type="radio" id="check-female" name="gender" value="female" />
            <label for="check-female">Female</label>
        </div>
    </div>
</div>
<div class="input-box address">
    <label>PickUp Address</label>
    <input type="text" placeholder="Enter street address" name="pickupAdd" required />
</div>
<div class="column">
    <div class="input-box">
        <label>Phone Number</label>
        <input type="number" placeholder="Enter phone number" name="phone" required
/>
    </div>
    <div class="input-box">
        <label>Date</label>
        <input type="date" placeholder="Enter date" name="date" required />
    </div>
</div>
<div class="column">
    <div class="input-box">
        <label>From Time</label>
        <input type="time" placeholder="Enter from time" name="from_time" required />
    </div>
    <div class="input-box">
        <label>To Time</label>
        <input type="time" placeholder="Enter to time" name="to_time" required />
    </div>
</div>

<div class="column">
    <div class="select-box">
        <label>From Where</label>
        <select name="from_station" required>
            <option value="" hidden>Select From</option>
            <option value="Vasai">Vasai</option>
            <option value="Naigaon">Naigaon</option>
            <option value="Bhayandar">Bhayandar</option>
            <option value="Miraroad">Mira Road</option>
        </select>
    </div>
    <div class="select-box">
```

```
            <label>To Where</label>
            <select name="to_station" required>
                <option value="" hidden>Select To</option>
                <option value="Vasai">Vasai</option>
                <option value="Naigaon">Naigaon</option>
                <option value="Bhayandar">Bhayandar</option>
                <option value="Miraroad">Mira Road</option>
            </select>
        </div>
    </div>
    <div class="column">
        <div class="input-box">
            <label>Number of Vehicle</label>
            <input type="number" placeholder="Enter vehicle number" name="vehicleNo"
required />
        </div>
        <div class="select-box input-box">
            <label>Types of Vehicle</label>
            <select name="wheeler_type" required>
                <option value="" hidden>Select Type</option>
                <option value="car">Car</option>
                <option value="motorcycle">Motorcycle</option>
                <option value="bicycle">Bicycle</option>
                <option value="truck">Truck</option>
            </select>
        </div>
    </div>
    <button>Submit</button>
    <button class="back-btn backButton" type="button" id="backBtn">Back</button>
  </form>
</section>
<section class="container formTwo dsp-none">
  <form class="form">
    <h3 class="registrationForm">Traveller Registration Form</h3></br></br>
    <div class="input-box">
        <label>Full Name</label>
        <input type="text" name="name" id="name" placeholder="Enter full name" />
    </div>
    <div class="input-box">
        <label>Email Address</label>
        <input type="text" name="email" id="senderMail" placeholder="Enter email address"
/>
    </div>
    <div class="gender-box">
        <h3>Gender</h3>
        <div class="gender-option">
            <div class="gender">
                <input type="radio" id="check-male-two" name="gender" id="genderMale"
value="male" />
                <label for="check-male">Male</label>
            </div>
```

```
            <div class="gender">
                <input type="radio" id="check-female-two" name="gender" id="genderFemale"
value="female" />
                <label for="check-female">Female</label>
            </div>
        </div>
    </div>
    <div class="column">
        <div class="input-box">
            <label>Phone Number</label>
            <input type="number" id="phoneNo" name="phoneNo" placeholder="Enter phone
number" />
        </div>
        <div class="input-box">
            <label>Date</label>
            <input type="date" id="birthDate" name="birthDate" placeholder="Enter birth
date" />
        </div>
    </div>

    <div class="column">
        <div class="input-box">
            <label>From Time</label>
            <input type="time" placeholder="Enter from time" name="fromTime"
id="fromTime" />
        </div>
        <div class="input-box">
            <label>To Time</label>
            <input type="time" placeholder="Enter to time" name="toTime" id="toTime" />
        </div>
    </div>

    <div class="column">
        <div class="select-box">
            <label>From Where</label>
            <select name="fromStation" id="fromStation" required>
                <option value="" hidden>Select From</option>
                <option value="Vasai">Vasai</option>
                <option value="Naigaon">Naigaon</option>
                <option value="Bhayandar">Bhayandar</option>
                <option value="Miraroad">Mira Road</option>
            </select>
        </div>
        <div class="select-box">
            <label>To Where</label>
            <select name="toStation" id="toStation" required>
                <option value="" hidden>Select To</option>
                <option value="Vasai">Vasai</option>
                <option value="Naigaon">Naigaon</option>
                <option value="Bhayandar">Bhayandar</option>
                <option value="Miraroad">Mira Road</option>
```

```
            </select>
          </div>
        </div>
        <button type="submit" id="travellerSubmitBtn">Submit</button>
        <button type="button" id="backButton" class="back-btn backButton">Back</button>
    </form>
</section>
<section class="formThree dsp-none">
    <div class="wrapper scroll-bottom">
        <div class="input-box-to">
          <select id="from_station" class="hidden-select" required>
            <option value="" hidden>Select To</option>
            <option value="Vasai">Vasai</option>
            <option value="Naigaon">Naigaon</option>
            <option value="Bhayandar">Bhayandar</option>
            <option value="Miraroad">Mira Road</option>
          </select>
        </div>
        <div class="input-box-to">
          <select id="to_station" class="hidden-select" required>
            <option value="" hidden>Select To</option>
            <option value="Vasai">Vasai</option>
            <option value="Naigaon">Naigaon</option>
            <option value="Bhayandar">Bhayandar</option>
            <option value="Miraroad">Mira Road</option>
          </select>
        </div>
        <div class="alert">
          <span class="text">Please Select Your Destination</span>
        </div>
        <div id="searchButton" class="input-box-to searchButton">
          <input type="button" value="Search">
        </div>
    </div>
</section>
<section class="heroSection">
    <section class="hero-section">
        <div class="hero">
          <h2>EasyGoHome Web for EasyGo</h2>
          <p>
            Verifying driver details, and sharing trip info. Be punctual and respectful
            to drivers, maintaining cleanliness. Have payment ready and use emergency
            features when necessary for a smooth EasyGo ride.
          </p>
          <div class="buttons">
            <a type="button" id="riderBtn" class="join">Driver</a>
            <a type="button" id="travellerBtn" class="learn">Traveller</a>
          </div>
        </div>
        <!--<div>
          <img src="image\mainBannerEasyGo.png" alt="hero image" />
```

```
        </div>-->
    </section>
</section>
<div id="selectRiderCard" class="selectRiderForm dsp-none">
    <ul class="sortable-list" id="detailsDiv">
    </ul>
</div>

<div id="id01" class="popUP_Modal" style="display:none;">
    <div class="modal-content">
        <div class="container">
            <h2 id="modalTitle">Message</h2></br>
            <p id="modalMessage"></p>

            <div class="clearfix">
                <button type="button"
onclick="document.getElementById('id01').style.display='none'"
class="deletebtn">Ok</button>
            </div>
        </div>
    </div>
</div>
<footer>
    <p>&copy; 2023 EasyGo - All rights reserved.</p>
</footer>
<script src="responseScript.js"></script>
<script>
function toggleMenu() {
        var menu = document.getElementById("myLinks");
        if (menu.style.display === "block") {
            menu.style.display = "none";
        } else {
            menu.style.display = "block";
        }
    }
</script>
</body>
</html>
```

3. **InsertDataSuccessfullyPage.html**

```
<html>
<head>
    <style>
* {
  box-sizing: border-box;
}

body {
  font-family: Arial, Helvetica, sans-serif;
}

/* Float four columns side by side */
.column {
  float: left;
  width: 25%;
  padding: 0 10px;
}

/* Remove extra left and right margins, due to padding */
.row {margin: 0 -5px;}

/* Clear floats after the columns */
.row:after {
  content: "";
  display: table;
  clear: both;
}

/* Responsive columns */
@media screen and (max-width: 600px) {
  .column {
    width: 100%;
    display: block;
    margin-bottom: 20px;
  }
}

/* Style the counter cards */
.card {
  box-shadow: 0 4px 8px 0 rgba(0, 0, 0, 0.2);
  padding: 16px;
  text-align: center;
  background-color: #f1f1f1;
}
</style>
</head>
<body>
<center><h2>Rider data insert Successfully!</h2></center>
```

```
<div id="data-table" class="column">
    <div class="card">
        <h3>Card 1</h3>
    </div>
</div>

</body>
<script src="responceScript.js"></script>
</html>
```

4. AboutUsPage.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>About Us - EasyGo</title>
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0-
beta3/css/all.min.css">
    <link rel="stylesheet" href="css/aboutus.css">
    <style>
        body {
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 0;
}

    .logo {
            position: absolute;
            top: 17px;
            left: 19px;
            color: white;
        }

        .logo a {
            color: white;
            text-decoration: none;
        }

        header {
            background-color: purple;
            color: white;
            text-align: center;
            padding: 3px 0;
            position: sticky;
            top: 0;
            z-index: 1000;
```

```
        }
         h1 {
            font-size: 20px;
        }

   section {
    max-width: 800px;
    margin: 20px auto;
    padding: 20px;
    border: 1px solid #ddd;
   }

.about-us h2,
.team h2,
.contact h2 {
    color: #333;
    border-bottom: 2px solid #333;
    padding-bottom: 10px;
}

.team-member {
    text-align: center;
    margin-bottom: 20px;
}

.team-member img {
    max-width: 100%;
    border-radius: 50%;
}

footer {
    background-color: purple;
    color: white;
    text-align: center;
    position: fixed;
    bottom: 0px;
    width: 100%;
}


    </style>
</head>
<body>
<header>
    <div class="logo">
        <a href="http://localhost:8050/EasyGoHome/"><i class="fas fa-home"></i> EasyGo
Home</a>
    </div>
    <h1>About EasyGo</h1>
</header>
<section class="about-us">
```

```
    <h2>Welcome to EasyGo</h2>
    <p>
        EasyGo is not just a transportation solution; it's a revolution in the way people move and connect. Established with a vision to make travel convenient, efficient, and enjoyable, EasyGo has been transforming the landscape of transportation since its inception.
    </p>

    <h2>Our Mission</h2>
    <p>
        At EasyGo, our mission is to provide a seamless and inclusive travel experience. We aim to connect people and places, making transportation easy and accessible for everyone. With a commitment to convenience, efficiency, and environmental sustainability, we strive to redefine the way people travel.
    </p>

    <h2>Our Values</h2>
    <p>
        EasyGo is built on core values that guide our every decision. We believe in accessibility for all, making sure our platform is user-friendly and caters to diverse needs. Sustainability is at the heart of our operations; we are dedicated to contributing to a greener world through innovative and eco-friendly transportation solutions.
    </p>

    <h2>Why Choose EasyGo?</h2>
    <p>
        EasyGo stands out in the transportation industry for its commitment to user satisfaction, reliability, and innovation. Our technology-driven approach ensures a hassle-free experience for commuters. Whether it's a daily commute or a long-distance journey, choosing EasyGo means choosing a journey that is not just a means of travel but an enriching experience.
    </p>

</section>
<section class="team">
    <h2>Owner of EasyGo</h2>
    <div class="team-member">
        <img src="images/team-member-1.jpg" alt="Team Member 1">
        <h3>Sunny Jaiswar</h3>
        <p>Founder & CEO</p>
    </div>
</section>
<footer>
    <p>&copy; 2023 EasyGo - All rights reserved.</p>
</footer>
</body>
</html>
```

5. CategoryPage.html

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>About Us - EasyGo</title>
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0-
beta3/css/all.min.css">
    <link rel="stylesheet" href="frontend/css/categorypage.css">
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            padding: 0;
            display: flex;
            flex-direction: column;
            height: 100vh;
        }

        .content {
            flex: 1;
            overflow-y: auto;
            padding: 20px;
        }

        .logo {
            position: absolute;
            top: 17px;
            left: 19px;
            color: white;
        }

        .logo a {
            color: white;
        }

        header {
            background-color: purple;
            color: white;
            text-align: center;
            padding: 3px 0;
            position: sticky;
            top: 0;
            z-index: 1000;
        }
         h1 {
            font-size: 20px;
        }

        section {
            max-width: 800px;
            margin: 20px auto;
            padding: 20px;
            border: 1px solid #ddd;
```

```
        }

        .about-us h2 {
            color: #333;
            border-bottom: 2px solid #333;
            padding-bottom: 10px;
        }

        .team-member {
            text-align: center;
            margin-bottom: 20px;
        }

        .team-member img {
            max-width: 100%;
            border-radius: 50%;
        }

    footer {
    background-color: purple;
    color: white;
    text-align: center;
    position: fixed;
    bottom: 0px;
    width: 100%;
    }
    </style>
</head>
<body>
<header>
    <div class="logo">
        <a href="http://localhost:8050/EasyGoHome/"><i class="fas fa-home"></i> EasyGo
Home</a>
    </div>
    <h1>There are two categories of EasyGo</h1>
</header>
<div class="content">
    <section class="about-us">
        <h2>First Category: Driver</h2>
        <p>
        <h3>Description:</h3>
        Riders are individuals who provide transportation services to Travellers. EasyGo's Rider
category empowers individuals with a vehicle to become drivers and earn income by offering
rides to those in need. Riders play a crucial role in creating a connected and accessible
transportation network within the EasyGo community.
        <h3>Driver Registration:</h3>
        Individuals interested in becoming Riders can easily register on the EasyGo platform,
showcasing their availability and vehicle details.
        <h3>Income Opportunities:</h3>
        Being a Rider provides an opportunity to earn income by offering rides to Travellers in
the vicinity.
```

```
        <h3>Flexible Schedule:</h3>
        Riders enjoy the flexibility of choosing their working hours and availability, allowing them to balance their personal and professional lives.
        </p>
    </section>
    <section class="about-us">
        <h2>Second Category: Traveller</h2>
        <p>
        <h3>Description:</h3>
        Travellers are individuals seeking transportation to reach their desired destinations. Whether it's a daily commute, a weekend getaway, or a long-distance journey, EasyGo's Traveller category is designed to provide a seamless and comfortable travel experience. Travellers can easily book rides, select their preferred routes, and enjoy the convenience of reliable transportation services.
        <h3>Booking Options:</h3>
        Travellers can choose from various booking options, including on-demand rides and scheduled trips.
        <h3>Route Flexibility:</h3>
        EasyGo offers route flexibility, allowing travellers to customize their journey based on their specific needs.
        <h3>Safety Measures:</h3>
        The platform prioritizes the safety and well-being of travellers, implementing safety features and standards for a secure ride.
        </p>
    </section>
</div>
<footer>
    <p>&copy; 2023 EasyGo - All rights reserved.</p>
</footer>
</body>
</html>
```

6.ContectUsPage.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>About Us - EasyGo</title>
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0-beta3/css/all.min.css">
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            padding: 0;
        }

        .logo {
            position: absolute;
```

```
        top: 17px;
        left: 19px;
        color: white;
    }

    .logo a {
        color: white;
    }

    header {
        background-color: purple;
        color: white;
        text-align: center;
        padding: 3px 0;
        position: sticky;
        top: 0;
        z-index: 1000;
    }
    h1 {
        font-size: 20px;
    }

    section {
        max-width: 800px;
        margin: 20px auto;
        padding: 20px;
        border: 1px solid #ddd;
    }

    .about-us h2,
    .team h2,
    .contact h2 {
        color: #333;
        border-bottom: 2px solid #333;
        padding-bottom: 10px;
    }

    .team-member {
        text-align: center;
        margin-bottom: 20px;
    }

    .team-member img {
        max-width: 100%;
        border-radius: 50%;
    }

footer {
background-color: purple;
color: white;
text-align: center;
```

```
    position: fixed;
    bottom: 0px;
    width: 100%;
    }
    </style>
</head>
<body>
<header>
    <div class="logo">
        <a href="http://localhost:8050/EasyGoHome/"><i class="fas fa-home"></i> EasyGo
Home</a>
    </div>
    <h1>About EasyGo</h1>
</header>
<section class="contact">
    <h2>Contact Us</h2>
    <p>If you have any questions or inquiries, feel free to reach out to us:</p>
    <p>Email: Sj3308098@gmail.com</p>
    <p>Phone: +91 8806-589-844</p>
</section>
<footer>
    <p>&copy; 2023 EasyGo - All rights reserved.</p>
</footer>
</body>
</html>
```

1.loginpage.css

```
/* Google Font Link login */
@import
url('https://fonts.googleapis.com/css2?family=Poppins:wght@200;300;400;500;600;700&display=sw
ap');
*{
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  font-family: "Poppins" , sans-serif;
}


/* Navigation */
nav{
 position: fixed;
 left: 0;
 top: 0;
 width: 100%;
 height: 55px;
 background-color: purple;
 box-shadow: 0 5px 10px rgba(0, 0, 0, 0.1);
}
nav .navbar{
 display: flex;
 align-items: center;
 justify-content: space-between;
 height: 100%;
 max-width: 90%;
 margin: auto;
}
nav .navbar .logo a{
  color: #fff;
  font-weight: 600;
}
nav .navbar .menu{
  display: flex;
}
.navbar .menu li{
  list-style:  none;
  margin: 0 15px;
}
.navbar .menu li a{
  color: #fff;
  font-size: 17px;
  font-weight: 500;
  text-decoration: none;
}
section{
  display: flex;
```

```
  height: 100vh;
  width: 100%;
  align-items: center;
  justify-content: center;
  color: #96c7e8;
  font-size: 70px;
}
#Home{
  background: #fff;
}
#About{
  background: #f2f2f2;
}
#Category{
  background: #e6e6e6;
}
#Latest{
  background: #fff;
}
#Contact{
  background: #f2f2f2;
}
#Feedback{
  background: #e6e6e6;
}
.button a{
  position: fixed;
  bottom: 20px;
  right: 20px;
  color: #fff;
  background: #2980b9;
  padding: 7px 12px;;
  font-size: 18px;
  border-radius: 6px;
  box-shadow: rgba(0, 0, 0, 0.15);
}

.nav-bar-middle{
    display: flex;
}

.dsp-none{
    display: none;
}

.loginBtn{
    font-size: 18px;
    color: purple;
    outline: none;
    border: none;
    border-radius: 5px;
```

```
    border: 2px solid antiquewhite;
    padding: 0px 13px;
    cursor: pointer;
}

/* Login Form */
body{
  min-height:   100vh;
  display:  inline-flex;
  align-items:   center;
  justify-content: end;
  padding: 30px;
  flex-direction: row-reverse;
}
.container {
   position: relative;
       max-width: 550px;
       width: 100%;
       background: #fff;
       padding: 1px 14px;
       box-shadow: 0 5px 10px rgba(0,0,0,0.2);
       perspective: 2700px;
       margin-left: 75px
}
.container .cover{
  position: absolute;
  top: 0;
  left: 50%;
  height: 100%;
  width: 50%;
  z-index: 98;
  transition: all 1s ease;
  transform-origin: left;
  transform-style: preserve-3d;
}
.container #flip:checked ~ .cover{
  transform: rotateY(-180deg);
}
 .container .cover .front,
 .container .cover .back{
  position: absolute;
  top: 0;
  left: 0;
  height: 100%;
  width: 100%;
}
.cover .back{
  transform: rotateY(180deg);
  backface-visibility: hidden;
}
.container .cover::before,
```

```
.container .cover::after{
  content: '';
  position: absolute;
  height: 100%;
  width: 100%;
  background: #7d2ae8;
  opacity: 0.5;
  z-index: 12;
}
.container .cover::after{
  opacity: 0.3;
  transform: rotateY(180deg);
  backface-visibility: hidden;
}
.container .cover img{
  position: absolute;
  height: 100%;
  width: 100%;
  object-fit: cover;
  z-index: 10;
}
.container .cover .text{
  position: absolute;
  z-index: 130;
  height: 100%;
  width: 100%;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
}
.cover .text .text-1,
.cover .text .text-2{
  font-size: 26px;
  font-weight: 600;
  color: #fff;
  text-align: center;
}
.cover .text .text-2{
  font-size: 15px;
  font-weight: 500;
}
.container .forms{
  height: 100%;
  width: 100%;
  background: #fff;
  margin-top: 45px;
}
.container .form-content{
  display: flex;
  align-items: center;
```

```
  justify-content: space-between;
}
.form-content .login-form,
.form-content .signup-form{
  width: calc(100% / 2 - 25px);
}
.forms .form-content .title{
  position: relative;
  font-size: 24px;
  font-weight: 500;
  color: #333;
}
.forms .form-content .title:before{
  content: '';
  position: absolute;
  left: 0;
  bottom: 0;
  height: 3px;
  width: 25px;
}
.forms .signup-form  .title:before{
  width: 20px;
}
.forms .form-content .input-boxes{
  margin-top: 10px;
}
.forms .form-content .input-box{
  display: flex;
  align-items: center;
  height: 50px;
  width: 100%;
  margin: 10px 0;
  position: relative;
}
.form-content .input-box input{
  height: 100%;
  width: 100%;
  outline: none;
  border: none;
  padding: 0 30px;
  font-size: 16px;
  font-weight: 500;
  border-bottom: 2px solid rgba(0,0,0,0.2);
  transition: all 0.3s ease;
}
.form-content .input-box input:focus,
.form-content .input-box input:valid{
  border-color: #7d2ae8;
}
.form-content .input-box i{
  position: absolute;
```

```
  font-size: 17px;
}
.forms .form-content .text{
  font-size: 14px;
  font-weight: 500;
  color: #333;
}
.forms .form-content .text a{
  text-decoration: none;
  color: purple;
}
.forms .form-content .text a:hover{
  text-decoration: underline;
}
.forms .form-content .button{
  color: #fff;
  margin-top: 40px;
}
.forms .form-content .button input{
  color: #fff;
  background: purple;
  border-radius: 6px;
  padding: 0;
  cursor: pointer;
  transition: all 0.4s ease;
}
.forms .form-content .button input:hover{
  background: #5b13b9;
}
.forms .form-content label{
  color: purple;
  cursor: pointer;
}
.forms .form-content label:hover{
  text-decoration: underline;
}
.forms .form-content .login-text,
.forms .form-content .sign-up-text{
  text-align: center;
  margin-top: 25px;
}
.container #flip{
  display: none;
}

ul li a:hover {
      color: black;
      border-bottom: 2px solid #333; /* Add a solid underline on hover */
 }
```

```
@media (max-width: 730px) {
  .container .cover{
    display: none;
  }
  .form-content .login-form,
  .form-content .signup-form{
    width: 100%;
  }
  .form-content .signup-form{
    display: none;
  }
  .container #flip:checked ~ .forms .signup-form{
    display: block;
  }
  .container #flip:checked ~ .forms .login-form{
    display: none;
  }

}

footer {
  background-color: purple;
  color: white;
  text-align: center;
  position: fixed;
  bottom: 0;
  width: 100%;
  left: 0px;
  padding: 10px;
}
```

2.popupmodal.css

```
/* The Modal (background) */
.popUP_Modal {
    display: none; /* Hidden by default */
    position: fixed; /* Stay in place */
    z-index: 1; /* Sit on top */
    left: 0;
    top: 0;
    width: 100%; /* Full width */
    height: 100%; /* Full height */
    overflow: auto; /* Enable scroll if needed */
    background-color: rgb(0,0,0); /* Fallback color */
    background-color: rgba(0,0,0,0.4); /* Black w/ opacity */
    padding-top: 60px;
}

/* Modal Content/Box */
.modal-content {
```

```
    background-color: #fefefe;
    margin: 5% auto; /* 15% from the top and centered */
    padding: 20px;
    border: 1px solid #888;
    width: 80%; /* Could be more or less, depending on screen size */
    max-width: 500px; /* Maximum width */
    border-radius: 10px; /* Rounded corners */
    box-shadow: 0 5px 15px rgba(0,0,0,0.3);
    animation-name: animatetop;
    animation-duration: 0.4s;
}

/* Add Animation */
@keyframes animatetop {
    from {top: -300px; opacity: 0}
    to {top: 0; opacity: 1}
}

/* Style the container */
.container {
    padding: 41px;
}

/* Clearfix (clear floats) */
.clearfix::after {
    content: "";
    clear: both;
    display: table;
}

/* Style the Cancel and Delete buttons */
.deletebtn {
    padding: 14px 20px;
    margin: 30px 0;
    border:  none;
    cursor: pointer;
    width: 45%;
    border-radius: 5px;
    font-size: 16px;
}

.deletebtn {
    background-color: purple;
    color: white;
}

/* Add some hover effects to buttons */
.deletebtn:hover {
    opacity: 0.8;
}
```

```
/* Responsive styling */
@media screen and (max-width: 768px) {
    .modal-content {
        width: 90%; /* Reduce width on smaller screens */
    }
    .deletebtn {
        width: 100%; /* Stack buttons on smaller screens */
        margin: 5px 0;
    }
}
body {
  font-family: Arial, Helvetica, sans-serif;
}

.mobile-container {
  max-width: 480px;
  margin: auto;
  background-color: #555;
  height: 500px;
  color: white;
  border-radius: 10px;
}


.topnav #myLinks {
  display: none;
}

.topnav a {
  color: white;
  padding: 17px 16px;
  text-decoration: none;
  font-size: 17px;
  display: block;
}

.topnav a.icon {
  display: block;
  position: absolute;
  right: 0;
  top: 0;
}



.active {
  background-color: #04AA6D;
  color: white;
}
```

```
.card {
    box-shadow: 0 8px 15px 0 rgba(0, 0, 0, 0.2);
    padding: 7px;
    width: 6%;
    background-color: #f2e6e675;
    margin-left: 94%;
    position: absolute;
    border-radius: 5px;
    }

.fonthamburger {
    font-size: 1em;
    margin-bottom: 1px;
}
```

3.riderTravellerForm.css

```
/* after login page all forms css */
    header {
      background-color: purple;
      color: white;
      text-align: center;
      padding: 12px 0;
      position: sticky;
      top: 0;
      z-index: 1000;
    }

    footer {
      background-color: purple;
      color: white;
      text-align: center;
      position: fixed;
      bottom: 0;
      width: 100%;
      left: 0px;
      padding: 13px;
    }

    .container {
      padding: 100px 383px 60px;
      text-align: center;
    }

    .dsp-none {
      display: none;
    }

    .hero img {
      max-width: 100%;
```

```
  height: auto;
}

.buttons a {
  display: inline-block;
  margin: 10px;
  padding: 10px 20px;
  color: #fff;
  text-decoration: none;
  border-radius: 5px;
  background-color: #800080;
}

.buttons a:hover {
  background-color: #4b004b;
}

.form  {
  margin: 0 7%;
  max-width: 500px;
  text-align: left;
  background-color: #fff;
  padding: 20px;
  border-radius: 5px;
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
}

.form header {
  font-size: 1.5em;
  margin-bottom: 20px;
}

.input-box,
.gender-box,
.select-box,
.column {
  margin-bottom: 20px;
}

.input-box label,
.gender-box h3,
.select-box label {
  display: block;
  margin-bottom: 5px;
}

.input-box input,
.select-box select {
  width: 100%;
  padding: 10px;
  box-sizing: border-box;
```

```
  border: 1px solid #ccc;
  border-radius: 5px;
}

.gender-option {
  display: flex;
  justify-content: space-between;
}

.gender-option .gender {
  flex: 1;
}

.gender-option .gender input {
  margin-right: 5px;
}

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  font-family: "Open Sans", sans-serif;
}

.hero-section {
  display: flex;
  /*justify-content: space-evenly;*/
  align-items: center;
  padding: 100px;
}

.hero-section .hero {
  max-width: 50%;
  color: black;
}

.hero h2 {
  font-size: 2.4rem;
  margin-bottom: 20px;
}

.hero p {
  font-size: 1.2rem;
  margin-bottom: 20px;
}

.hero-section .buttons {
  margin-top: 40px;
}

.hero-section .buttons a {
```

```
  text-decoration: none;
  border: 1px solid black;
  color: black;
  padding: 12px 24px;
  border-radius: 0.375rem;
  font-weight: 600;
  transition: 0.2s ease;
  display: inline-block;
  color: white;
}

.hero-section .buttons a:not(:last-child) {
  margin-right: 15px;
  color: white;
}

.hero-section .buttons .learn {
  border: 1px solid black;
  border-radius: 0.375rem;
}

.hero-section .buttons a:hover {
  background-color: #BF40BF;
}

.input-box-to {
  margin-bottom: 20px;
}

.alert {
  margin-bottom: 20px;
  color: red;
}

/* .back-btn {
  background-color: #ccc;
  border: none;
  padding: 10px 20px;
  cursor: pointer;
}*/

.back-btn:hover {
  background-color: #aaa;
}

.searchButton input {
  background-color: #800080;
  color: #fff;
  border: none;
  padding: 10px 20px;
  border-radius: 5px;
```

```
  cursor: pointer;
}

.searchButton input:hover {
  background-color: #4b004b;
}

.registrationForm {
  background: purple;
  font-size: 20px;
  color: white;
  text-align: center;
  padding: 7px;
}


.form button {
  height: 35px;
  width: 30%;
  color: #fff;
  font-weight: 400;
  margin-top: 30px;
  border: none;
  cursor: pointer;
  transition: all 0.2s ease;
  background: #ab47bc;
}

.backButton {
  margin-left: 178px;
}

.form button:hover {
  background: rgb(88, 56, 250);
}

/*select rider for ride card */
@import
url('https://fonts.googleapis.com/css2?family=Poppins:wght@400;500;600&display=swap');

.sortable-list {
  background: #fff;
  border-radius: 7px;
  padding: 30px 25px 20px;
  box-shadow: 0 15px 30px rgba(0, 0, 0, 0.1);
  z-index: 5;
}

.sortable-list .item {
  list-style: none;
  display: flex;
```

```
  background: #fff;
  align-items: center;
  border-radius: 5px;
  padding: 10px 13px;
  margin-bottom: 11px;
  /* box-shadow: 0 2px 4px rgba(0,0,0,0.06); */
  border: 1px solid #ccc;
  justify-content: space-between;
}

.selectRiderForm {
  float: right;
  margin-left: 20px;
  height: 90vh;
  overflow-y: scroll;
  margin-top: -350px;
}

/*cards css*/
/* Add spacing between cards and make sure three cards fit in a row */
.rows {
  display: flex;
  flex-wrap: wrap;
  justify-content: space-between;
}

/* Adjust the width of each card to fit three cards in a row */
.columns {
  flex: 0 0 calc(33.33% - 10px);
  /* Adjust the width calculation for three cards with spacing */
  margin: 5px;
  /* Add spacing between cards */
  box-sizing: border-box;
}

/* Style the counter cards */
.card {
  box-shadow: 0 4px 8px 0 rgba(0, 0, 0, 0.2);
  padding: 16px;
  text-align: center;
  background-color: #f1f1f1;
}



/* formThree css */
/* Hamburger menu styles */
@import
url('https://fonts.googleapis.com/css2?family=Poppins:wght@200;300;400;500;600;700&display=sw
ap');
```

```
.wrapper {
  position: relative;
  background: #fff;
  max-width: 350px;
  width: 100%;
  padding: 35px 40px;
  border-radius: 6px;
  box-shadow: 0 5px 10px rgba(0, 0, 0, 0.2);
  left: 230px;
  top: 150px;
}

.input-box-to {
  position: relative;
  height: 40px;
  margin: 28px 10px;

}

.input-box-to input {
  position: relative;
  height: 100%;
  width: 50%;
  margin-right: 70px;
  outline: none;
  font-weight: 500;
  padding: 0 40px 0 16px;
}

.input-box-to label {
  position: absolute;
  top: 50%;
  transform: translateY(-50%);
  color: #a6a6a6;
  transition: all 0.3s ease;
  left: 15px;
  font-size: 18px;
  font-weight: 400;
  background: #fff;
  padding: 0 6px;
  pointer-events: none;
}

.input-box-to input:focus~label,
.input-box-to input:valid~label {
  top: 0;
  font-size: 14px;
  color: #4070F4;
}

.text {
```

```
  font-size: 18px;
  font-weight: 400;
  color: #a6a6a6;
  margin: 7px;
}

.input-box-to.searchButton input {
  font-size: 20px;
  color: #fff;
  letter-spacing: 1px;
  background: purple;
  left: 65px;
  cursor: pointer;

}

.hidden-select {
  position: relative;
  width: 95%;
  height: 100%;
  z-index: 1;
  cursor: pointer;
}
```

3. JAVASCRIPT

Package hierarchy: EasyGoHome→Src→Main→Webapp→FrontEnd

1.responseScript.js

```
/*show and haide all forms by id start methods here*/
    function riderButtonForm() {
            document.querySelector('.formOne').classList.remove('dsp-none');
            document.querySelector('.formTwo').classList.add('dsp-none');
            document.querySelector('.heroSection').classList.add('dsp-none');
        }

        document.getElementById('riderBtn').addEventListener('click', function (e) {
            riderButtonForm();
            localStorage.setItem('whichFormOpen', 'riderButtonForm');
        });

        function travellerBtnForm() {
            document.querySelector('.formThree').classList.remove('dsp-none');
            document.querySelector('.formTwo').classList.add('dsp-none');
            document.querySelector('.formOne').classList.add('dsp-none');
            document.querySelector('.heroSection').classList.add('dsp-none');
        }

        document.getElementById('travellerBtn').addEventListener('click', function (e) {
```

```
        travellerBtnForm();
        localStorage.setItem('whichFormOpen', 'travellerBtnForm');
    });

    function searchButtonForm() {
        document.querySelector('.selectRiderForm').classList.remove('dsp-none');
    }

    document.getElementById('searchButton').addEventListener('click', function (e) {
        searchButtonForm();
        localStorage.setItem('whichFormOpen', 'searchButtonForm');
    });

    function selectRiderCardForm() {
        document.querySelector('.selectRiderForm').classList.add('dsp-none');
        document.querySelector('.formThree').classList.add('dsp-none');
        document.querySelector('.formTwo').classList.remove('dsp-none');
    }

    document.getElementById('selectRiderCard').addEventListener('click', function (e) {
        selectRiderCardForm();
        localStorage.setItem('whichFormOpen', 'selectRiderCard');
    });

    document.getElementById('backBtn').addEventListener('click', function (e) {
        document.querySelector('.formOne').classList.add('dsp-none');
        document.querySelector('.formTwo').classList.add('dsp-none');
        document.querySelector('.heroSection').classList.remove('dsp-none');
        localStorage.removeItem('whichFormOpen');
    });

    document.getElementById('backButton').addEventListener('click', function (e) {
        document.querySelector('.formOne').classList.add('dsp-none');
        document.querySelector('.formTwo').classList.add('dsp-none');
        document.querySelector('.heroSection').classList.remove('dsp-none');
        localStorage.removeItem('whichFormOpen');
    });
/*show and haide all forms by id end methods here*/




/* get data of rider by searching to and from api call start */
function myFunction(e) {
  // Get values from input fields
  var fromStation = document.getElementById("from_station").value;
  var toStation = document.getElementById("to_station").value;
  //var url = "http://localhost:8050/EasyGoHome/riderData?fromStation=" +
encodeURIComponent(fromStation) + "&toStation=" + encodeURIComponent(toStation);

  var url = `http://localhost:${window.location.port}/EasyGoHome/riderData?fromStation=` +
```

```
encodeURIComponent(fromStation) + "&toStation=" + encodeURIComponent(toStation);
  var xhr = new XMLHttpRequest();

  // Set up the event listener to handle the response
  xhr.addEventListener("readystatechange", function() {
  if (this.readyState === 4) {
      if (this.status === 200) {
        try {

          var responseArray = JSON.parse(this.responseText);
          console.log(responseArray);
          populateTable(responseArray);
        } catch (error) {
          console.error("Error parsing JSON response:", error);
          console.log("Response Text:", this.responseText); // Log the response text for debugging
        }
      } else {
        console.error("Request failed with status: " + this.status);
      }
    }
    e.preventDefault();
  });

  xhr.open("POST", url);
  xhr.setRequestHeader("Content-Type", "application/json");
  xhr.send();
}

const submitButton = document.getElementById("searchButton");
submitButton.addEventListener("click", myFunction);

function populateTable(data) {
  const detailsDiv = document.getElementById("detailsDiv");

      // Clear existing content in detailsDiv
      detailsDiv.innerHTML = "";

      // Loop through the API data and create list items for each item
      data.forEach(item => {
          const listItem = document.createElement("li");
          listItem.classList.add("item");

          const details = document.createElement("div");
          details.classList.add("details");

          // Populate details with API data (modify this as per your data structure)
          details.innerHTML = `
              <h3>${item.name}</h3>
              <p>Phone: ${item.phone}</p>
              <p>Date: ${item.date}</p>
              <p>Email: ${item.email}</p>
```

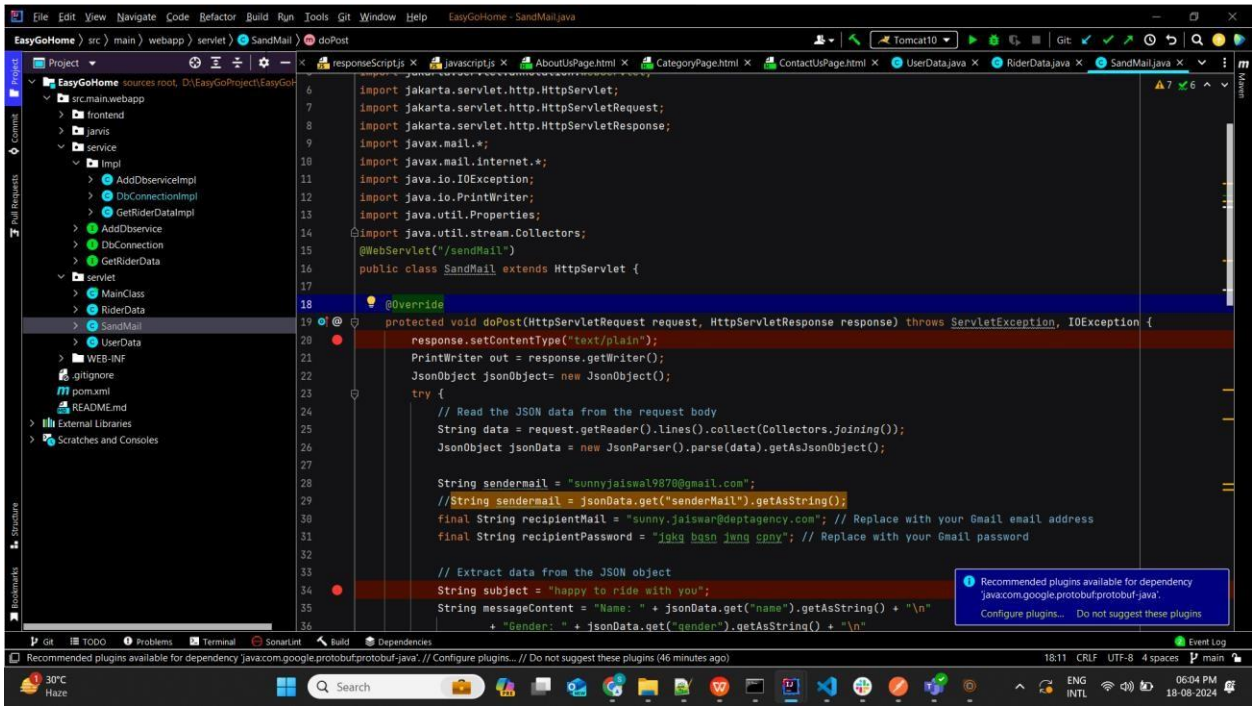
```
            <p>Gender: ${item.gender}</p>
            <p>From: ${item.from}</p>
            <p>To: ${item.to}</p>
            <p>From Time: ${item.fromTime}</p>
            <p>To Time: ${item.toTime}</p>
            <p>Wheeler: ${item.wheeler}</p>
        `;
    details.addEventListener("click", () => {
    showModal(`${item.name}`,"want to ride with Above Rider then please fill the form and send
mail");
    });
        // Append details to the list item and list item to the detailsDiv
        listItem.appendChild(details);
        detailsDiv.appendChild(listItem);
     });
  }
/* get data of rider by searching to and from api call end */

/* send mail to rider by traveller api integration start */
function sendMailFunction(e) {
    e.preventDefault(); // Prevent the default form submission behavior

    // Get values from input fields
    var name = document.getElementById("name").value;
    var senderMail = document.getElementById("senderMail").value;
    var gender = document.querySelector('input[name="gender"]:checked').value;
    var phoneNo = document.getElementById("phoneNo").value;
    var birthDate = document.getElementById("birthDate").value;
    var fromTime = document.getElementById("fromTime").value;
    var toTime = document.getElementById("toTime").value;
    var fromStation = document.getElementById("fromStation").value;
    var toStation = document.getElementById("toStation").value;

    // Validate form data (add your validation logic)

    var data = {
        name: name,
        senderMail: senderMail,
        gender: gender,
        phoneNo: phoneNo,
        birthDate: birthDate,
        fromTime: fromTime,
        toTime: toTime,
        fromStation: fromStation,
        toStation: toStation
    };

    var apiUrl = "http://localhost:8050/EasyGoHome/sendMail";

    var xhr = new XMLHttpRequest();
    var jsonData = JSON.stringify(data);
```

```
    xhr.addEventListener("load", function () {
        if (xhr.status === 200) {
            var response = JSON.parse(xhr.responseText);
            console.log("Response from server:", response);

            // Handle the response here, e.g., update the UI
            if (response.status === "success") {
                showModal("Success", "Email sent successfully! Driver will Contact You Soon ");
            } else {
                showModal("Failure", "Failed to send email: " + response.message);
            }
        } else {
            console.error("Mail send failed with status:", xhr.status);
        }
    });

    xhr.open("POST", apiUrl, true);
    xhr.setRequestHeader("Content-Type", "application/json");
    xhr.send(jsonData);
}

var travellerSubmitBtn = document.getElementById("travellerSubmitBtn");
travellerSubmitBtn.addEventListener("click", sendMailFunction);

/* send mail to rider by traveller api integration end */

//start calling popup model
function showModal(title, message) {
    document.getElementById('modalTitle').textContent = title;
    document.getElementById('modalMessage').textContent = message;
    document.getElementById('id01').style.display = 'block';
}
```

2.javascript.js

```
document.getElementById('loginForm').addEventListener('click', function (e) {
    document.querySelector('.login-signup_form').classList.remove('dsp-none');
})
//close login form
document.querySelector('#close').addEventListener('click', function (e) {
    document.querySelector('.login-signup_form').classList.add('dsp-none');

})
```

4. BACKEND :CORE JAVA, ADV. JAVA(SERVLET,JDBC)



1.Package hierarchy: EasyGoHome→Src→Main→Webapp→Servlet

1.userData (API)

```
package src.main.webapp.servlet;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.osgi.service.component.annotations.Reference;
import src.main.webapp.service.AddDbservice;
import src.main.webapp.service.Impl.AddDbserviceImpl;
import java.io.IOException;
import java.io.PrintWriter;
import java.sql.*;
@WebServlet("/userDetail")
public class UserData extends HttpServlet {

    @Reference
    AddDbservice addDbservice;

    public void init() {
        addDbservice = new AddDbserviceImpl();
    }
```

```
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
        PrintWriter out = response.getWriter();
        out.println("welcome in EasyGo");
        String name = request.getParameter("n1");
        String password = request.getParameter("p1");
        String emailId = request.getParameter("e1");
        out.println("hello ," + name);
        out.println("passowrd is ," + password);

        boolean loginSuccessful = false;
        try {
            loginSuccessful = addDbservice.addProduct(name,password,emailId);
        } catch (SQLException | ClassNotFoundException e) {
            e.printStackTrace();
        }
        if (loginSuccessful) {
            // If SignUp Form is successful, redirect to the login successful page
            response.sendRedirect("Rider.html");
        } else {
            // If SignUp Form, redirect back to the login page with an error message (optional)
            response.sendRedirect("LoginFail.html");
        }
    }

    //login page
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
        PrintWriter out = response.getWriter();
        String userEmail = request.getParameter("e1");
        String enteredPassword = request.getParameter("p1");

        try {

            Class.forName("com.mysql.cj.jdbc.Driver");

            // Step 2: Establish a connection to the database
            String url = "jdbc:mysql://127.0.0.1:3306/student?useSSL=false";
            String username = "root";
            String password = "Sunny@123";
            Connection connection = DriverManager.getConnection(url, username, password);

            // Step 3: Create a SQL query to retrieve the user data based on the entered username
            String sqlQuery = "SELECT password FROM users WHERE email = ?";
            PreparedStatement preparedStatement = connection.prepareStatement(sqlQuery);
            preparedStatement.setString(1, userEmail);

            // Step 4: Execute the query and check if the user exists in the database
            ResultSet resultSet = preparedStatement.executeQuery();
```

```
            if (resultSet.next()) {
                // Retrieve the stored password from the database
                String storedPassword = resultSet.getString("password");

                // Step 5: Compare the entered password with the stored password
                if (storedPassword.equals(enteredPassword)) {
                    response.sendRedirect("Rider.html");
                } else {
                    response.sendRedirect("LoginFail.html");
                }
            } else {
                response.sendRedirect("LoginFail.html");
            }

            // Close the resources
            resultSet.close();
            preparedStatement.close();
            connection.close();

        } catch (ClassNotFoundException | SQLException e) {
            e.printStackTrace();
            out.println("<h2>Error: Unable to check login credentials.</h2>");
        }
    }
}
```

2.RidderData(API)

```
package src.main.webapp.servlet;


import com.google.gson.JsonArray;
import com.google.gson.JsonObject;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.osgi.service.component.annotations.Reference;
import src.main.webapp.service.GetRiderData;
import src.main.webapp.service.Impl.GetRiderDataImpl;

import java.io.IOException;
import java.io.PrintWriter;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
```

```
@WebServlet("/riderData")
public class RiderData extends HttpServlet {

    @Reference
    GetRiderData getRiderData;

    public void init(){
        getRiderData = new GetRiderDataImpl();
    }
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
        PrintWriter out = response.getWriter();
        out.println("this is get rider Data");

        String name = request.getParameter("name");
        String phone = request.getParameter("phone");
        String date = request.getParameter("date");
        String from_time = request.getParameter("from_time");
        String to_time = request.getParameter("to_time");
        String email = request.getParameter("email");
        String gender = request.getParameter("gender");
        String from_station = request.getParameter("from_station");
        String to_station = request.getParameter("to_station");
        String wheeler = request.getParameter("wheeler_type");

        boolean RiderDataSuccess = false;
        try {
            RiderDataSuccess = getRiderData.getRiderDetail(name, phone, date, from_time, to_time,
email,gender, from_station, to_station,wheeler);

        }catch (Exception e){
            e.printStackTrace();
        }
        if (RiderDataSuccess) {
            // If SignUp Form is successful, redirect to the login successful page
            response.sendRedirect("InsertDataSuccessPage.html");
        } else {
            // If SignUp Form, redirect back to the login page with an error message (optional)
            response.sendRedirect("login.html?error=1");
        }


    }

    //Traveller Data fetch

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
```

```
        PrintWriter out = response.getWriter();
        response.setContentType("application/json");
        JsonArray jsonArray = new JsonArray();
        String fromStation = request.getParameter("fromStation");
        String toStation = request.getParameter("toStation");

        try {

            Class.forName("com.mysql.cj.jdbc.Driver");

            // Step 2: Establish a connection to the database
            String url = "jdbc:mysql://127.0.0.1:3306/student?useSSL=false";
            String username = "root";
            String password = "Sunny@123";
            Connection connection = DriverManager.getConnection(url, username, password);

            // Step 3: Create a SQL query to retrieve the user data based on the entered username
            String sqlQuery = "SELECT * FROM rider WHERE fromStation = ? AND toStation = ?";
            PreparedStatement preparedStatement = connection.prepareStatement(sqlQuery);
            preparedStatement.setString(1,fromStation);
            preparedStatement.setString(2,toStation);

            // Step 4: Execute the query and check if the station exists in the database
            ResultSet resultSet = preparedStatement.executeQuery();

            while (resultSet.next()) {
                JsonObject jsonObject = new JsonObject();
                jsonObject.addProperty("name", resultSet.getString("userName"));
                jsonObject.addProperty("phone", resultSet.getString("userPhone"));
                jsonObject.addProperty("date", resultSet.getString("currentDate"));
                jsonObject.addProperty("email", resultSet.getString("userEmail"));
                jsonObject.addProperty("gender", resultSet.getString("userGender"));
                jsonObject.addProperty("from", resultSet.getString("fromStation"));
                jsonObject.addProperty("to", resultSet.getString("toStation"));
                jsonObject.addProperty("fromTime", resultSet.getString("fromTime"));
                jsonObject.addProperty("toTime", resultSet.getString("toTime"));
                jsonObject.addProperty("wheeler", resultSet.getString("typeOfWheeler"));
                jsonArray.add(jsonObject);

            }

            out.println(jsonArray);


        }catch (Exception e){
            e.printStackTrace();
        }

        }
    }
```

3. SendMail(API)

```
package src.main.webapp.servlet;
import com.google.gson.JsonObject;
import com.google.gson.JsonParser;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import javax.mail.*;
import javax.mail.internet.*;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.Properties;
import java.util.stream.Collectors;
@WebServlet("/sendMail")
public class SandMail extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
        response.setContentType("text/plain");
        PrintWriter out = response.getWriter();
        JsonObject jsonObject= new JsonObject();
        try {
            // Read the JSON data from the request body
            String data = request.getReader().lines().collect(Collectors.joining());
            JsonObject jsonData = new JsonParser().parse(data).getAsJsonObject();

            String sendermail = "sunnyjaiswal9870@gmail.com";
            //String sendermail = jsonData.get("senderMail").getAsString();
            final String recipientMail = "sunny.jaiswar@gency.com"; // Replace with your Gmail email
address
            final String recipientPassword = "bqsn jwnq cpny"; // Replace with your Gmail password

            // Extract data from the JSON object
            String subject = "happy to ride with you";
            String messageContent = "Name: " + jsonData.get("name").getAsString() + "\n"
                    + "Gender: " + jsonData.get("gender").getAsString() + "\n"
                    + "Phone Number: " + jsonData.get("phoneNo").getAsString() + "\n"
                    + "Birth Date: " + jsonData.get("birthDate").getAsString();

            // The rest of your email sending code remains unchanged

            // Set up the mail server properties
            Properties props = new Properties();
            props.put("mail.smtp.host", "smtp.gmail.com");
            props.put("mail.smtp.port", "587");
            props.put("mail.smtp.auth", "true");
            props.put("mail.smtp.starttls.enable", "true");
```

```
        // Create a mail session with authentication
        Session session = Session.getInstance(props, new Authenticator() {
            protected PasswordAuthentication getPasswordAuthentication() {
                return new PasswordAuthentication(recipientMail, recipientPassword);
            }
        });

        try {
            // Create a message
            Message message = new MimeMessage(session);
            message.setFrom(new InternetAddress(recipientMail));
            message.setRecipient(Message.RecipientType.TO, new InternetAddress(sendermail));
            message.setSubject(subject);
            message.setText(messageContent);

            // Send the message using Transport
            Transport.send(message);
            jsonObject.addProperty("y", "Y");
            jsonObject.addProperty("status", "success");
            jsonObject.addProperty("message", "Email sent successfully!");

        } catch (MessagingException e) {
            e.printStackTrace();
            jsonObject.addProperty("status", "error");
            jsonObject.addProperty("message", "Failed to send email.");
        }
    }catch (Exception e){
        e.printStackTrace();
        jsonObject.addProperty("status", "error");
        jsonObject.addProperty("message", "Failed to send email.");
    }

    out.println(jsonObject);
}
}
```

2.JDBC(Java DataBase Connectivity)

1.Package hierarchy: EasyGoHome→Src→Main→Webapp→Service

1.AddDbservice

```
package src.main.webapp.service.Impl;

import org.osgi.service.component.annotations.Component;
import src.main.webapp.service.AddDbservice;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

@Component(immediate = true, service = AddDbservice.class)
public class AddDbserviceImpl implements AddDbservice {

    @Override
    public boolean addProduct(String name,String password,String email) throws SQLException,
ClassNotFoundException {

        String userName = name;
        String userPass = password;
        String userEmail = email;

        String sql_query = "INSERT INTO users (username, password,email) VALUES (?, ?, ?)";
        Class.forName("com.mysql.cj.jdbc.Driver");
        try (Connection connection =
DriverManager.getConnection("jdbc:mysql://127.0.0.1:3306/student?useSSL=false", "root",
"Sungr@12767");
            PreparedStatement preparedStatement = connection.prepareStatement(sql_query)) {
            preparedStatement.setString(1, userName);
            preparedStatement.setString(2, userPass);
            preparedStatement.setString(3, userEmail);


            int rowsAffected = preparedStatement.executeUpdate();
            return rowsAffected > 0;
        }
    }
}
```

2. GetRiderDataService

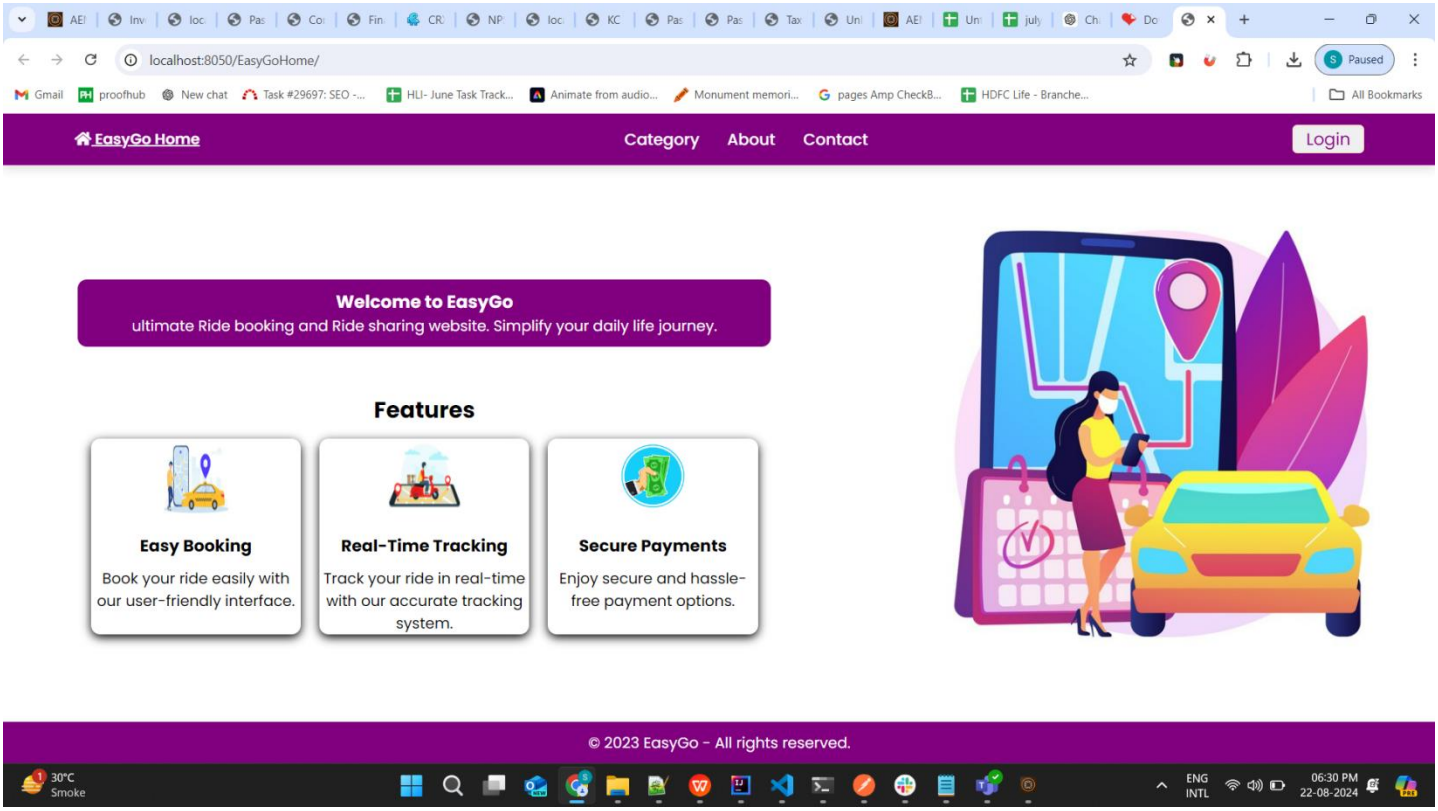
```
package src.main.webapp.service;

import java.sql.SQLException;

public interface GetRiderData {
   public boolean getRiderDetail(String name,String phone,String date,String from_time,String
to_time,String email,String gender,String from_station,String to_station,String wheeler) throws
SQLException, ClassNotFoundException;
}



package src.main.webapp.service.Impl;

import org.osgi.service.component.annotations.Component;
import src.main.webapp.service.GetRiderData;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

@Component(service = GetRiderData.class, immediate = true)
public class GetRiderDataImpl implements GetRiderData {

   @Override
   public boolean getRiderDetail(String name, String phone, String date, String from_time, String
to_time, String email,String gender, String from_station, String to_station,String wheeler) throws
SQLException, ClassNotFoundException {
      String userName = name;
      String userPhone = phone;
      String currentDate = date;
      String fromTime = from_time;
      String toTime = to_time;
      String userEmail = email;
      String userGender = gender;
      String fromStation = from_station;
      String toStation = to_station;
      String wheeler_type = wheeler;

      String sql_query = "INSERT INTO rider (username,
userPhone,currentDate,fromTime,toTime,userEmail,userGender,fromStation,toStation,typeOfWheele
r) VALUES (?, ?, ?,?, ?, ?,?, ?,?,?)";
      Class.forName("com.mysql.cj.jdbc.Driver");
      try (Connection connection =
DriverManager.getConnection("jdbc:mysql://127.0.0.1:3306/student?useSSL=false", "root",
"Sunny@123");
         PreparedStatement preparedStatement = connection.prepareStatement(sql_query)) {
         preparedStatement.setString(1, userName);
```

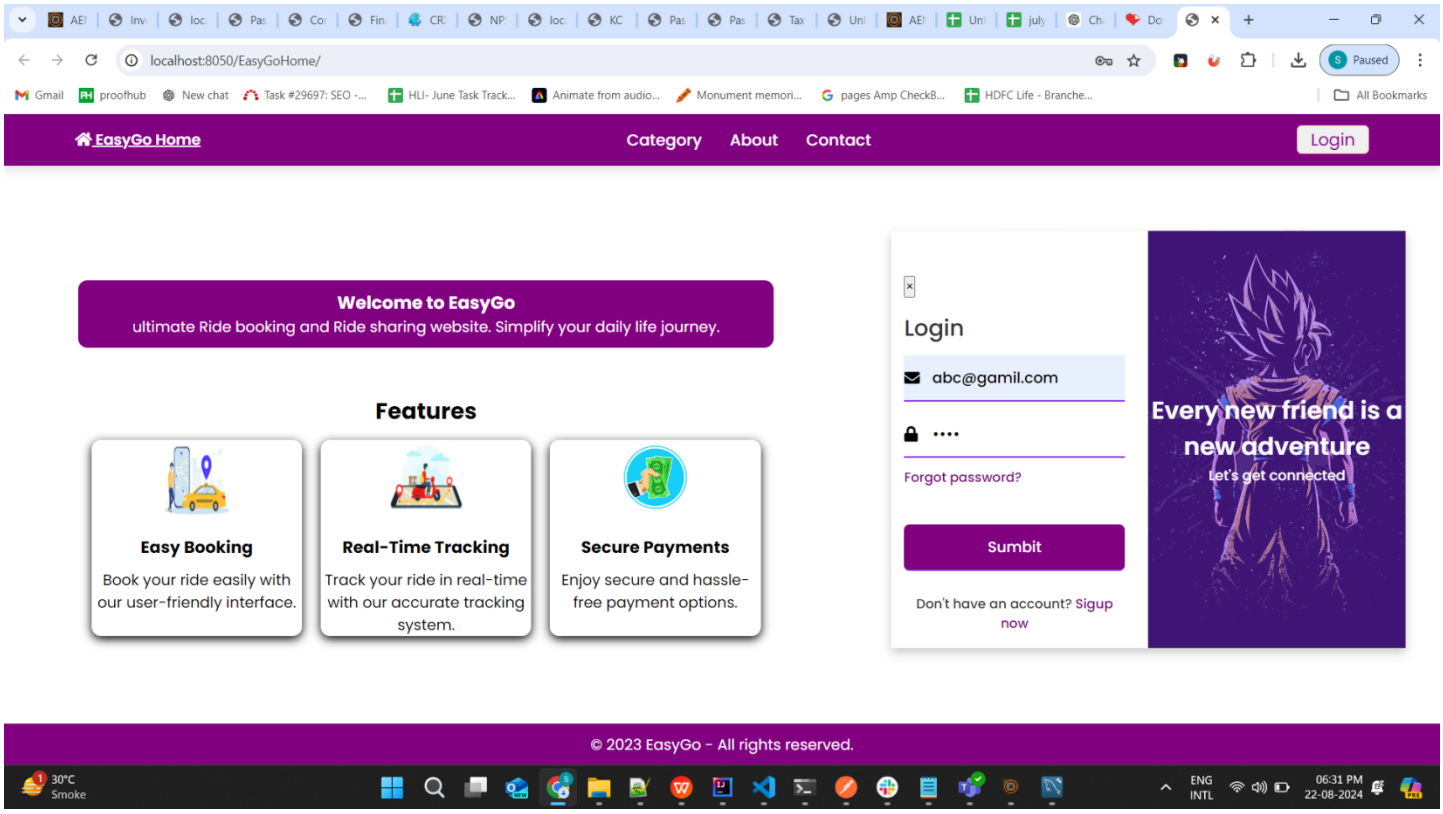
```
            preparedStatement.setString(2, userPhone);
            preparedStatement.setString(3, currentDate);
            preparedStatement.setString(4, fromTime);
            preparedStatement.setString(5, toTime);
            preparedStatement.setString(6, userEmail);
            preparedStatement.setString(7, userGender);
            preparedStatement.setString(8, fromStation);
            preparedStatement.setString(9, toStation);
            preparedStatement.setString(10, wheeler_type);


            int rowsAffected = preparedStatement.executeUpdate();
            return rowsAffected > 0;
        }
    }
}
```

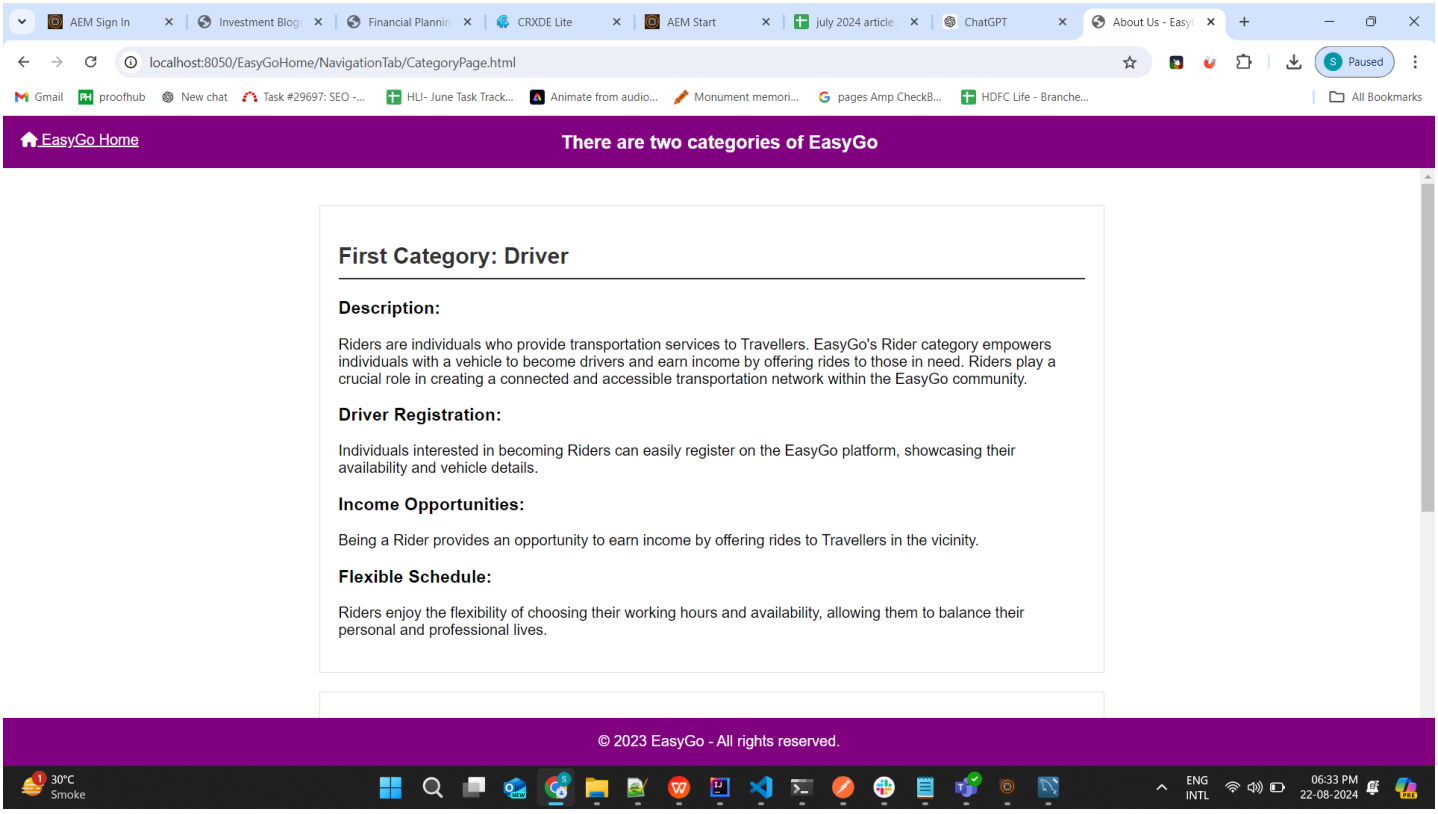
3. DbConnectionService

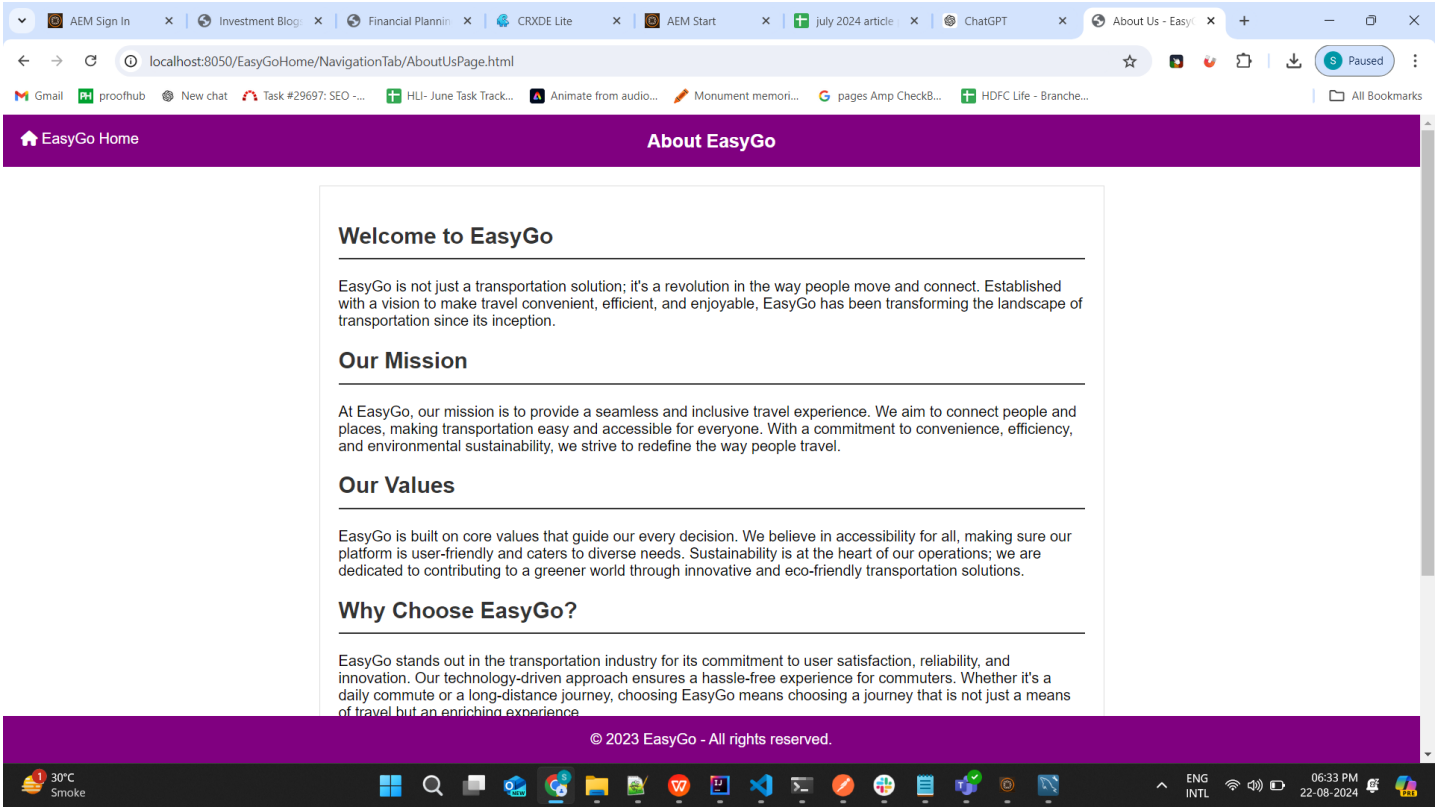
```
package src.main.webapp.service;

import java.sql.Connection;
import java.sql.SQLException;

public interface DbConnection {
    Connection getconnection() throws ClassNotFoundException, SQLException;

}

package src.main.webapp.service.Impl; // Adjust the package name to match your project structure

import org.osgi.service.component.annotations.Component;
import src.main.webapp.service.DbConnection;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

@Component(service = DbConnection.class)
public class DbConnectionImpl implements DbConnection {

    @Override
    public Connection getconnection() throws ClassNotFoundException, SQLException {
        // Load the MySQL JDBC driver
        Class.forName("com.mysql.cj.jdbc.Driver");
// Create a new connection to the database
        return DriverManager.getConnection(url, username, password);
    }
}
```

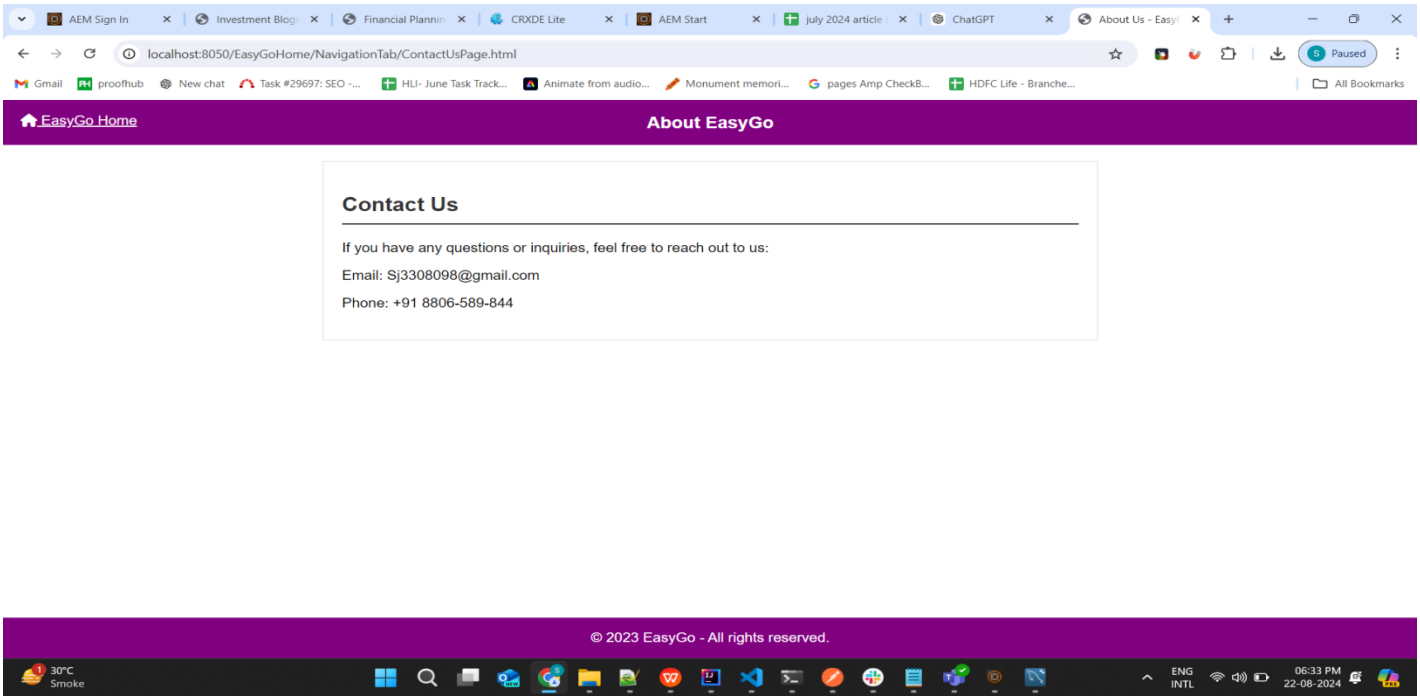
OUTPUT SCREENS

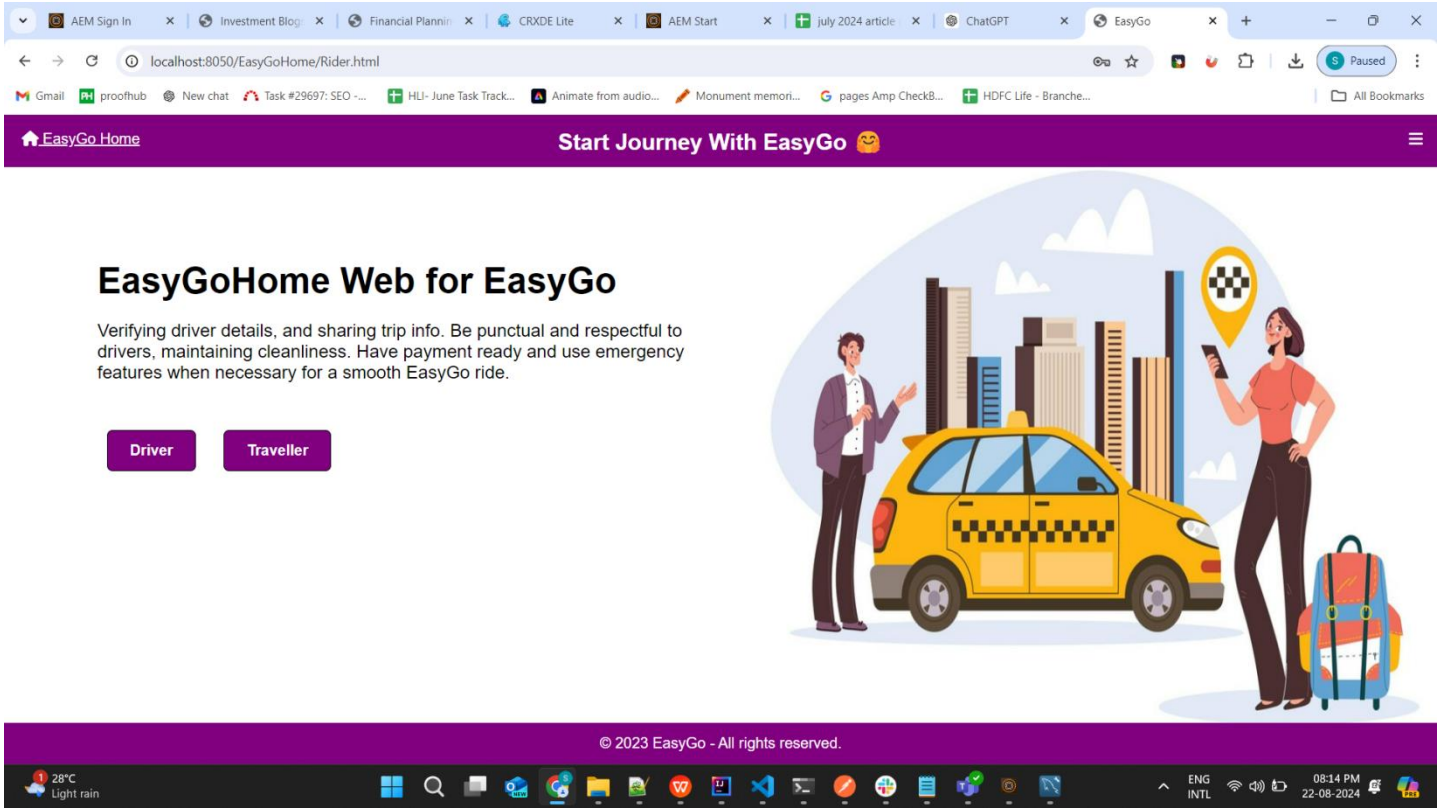


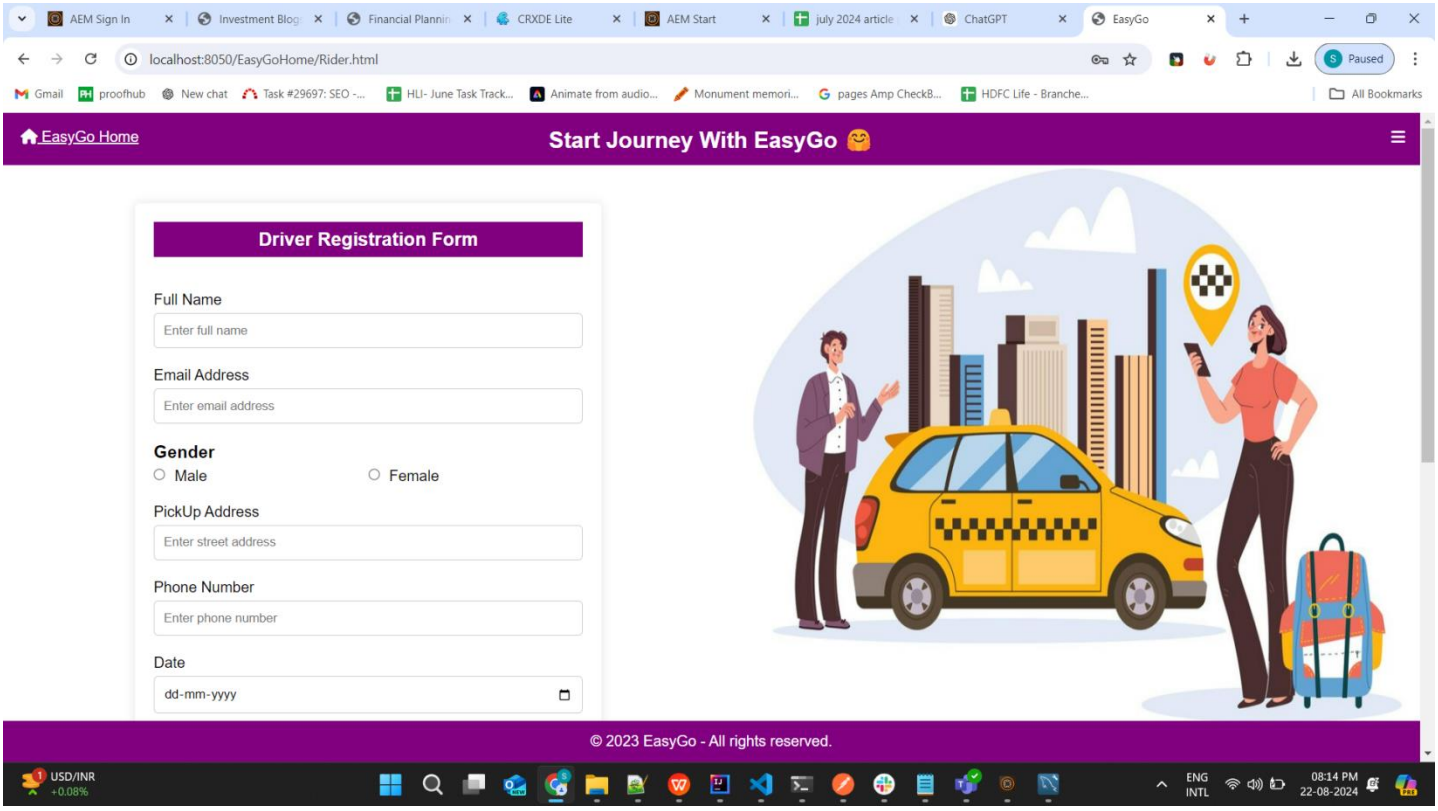

localhost:8050/EasyGoHome/
EasyGo Home
Category
About
Contact
Login
Welcome to EasyGo
ultimate Ride booking and Ride sharing website. Simplify your daily life journey.
Features
Easy Booking
Book your ride easily with our user-friendly interface.
Real-Time Tracking
Track your ride in real-time with our accurate tracking system.
Secure Payments
Enjoy secure and hassle-free payment options.
Login
abc@gamil.com
Forgot password?
Sumbit
Don't have an account? Sigup now
Every new friend is a new adventure
Let's get connected
© 2023 EasyGo - All rights reserved.

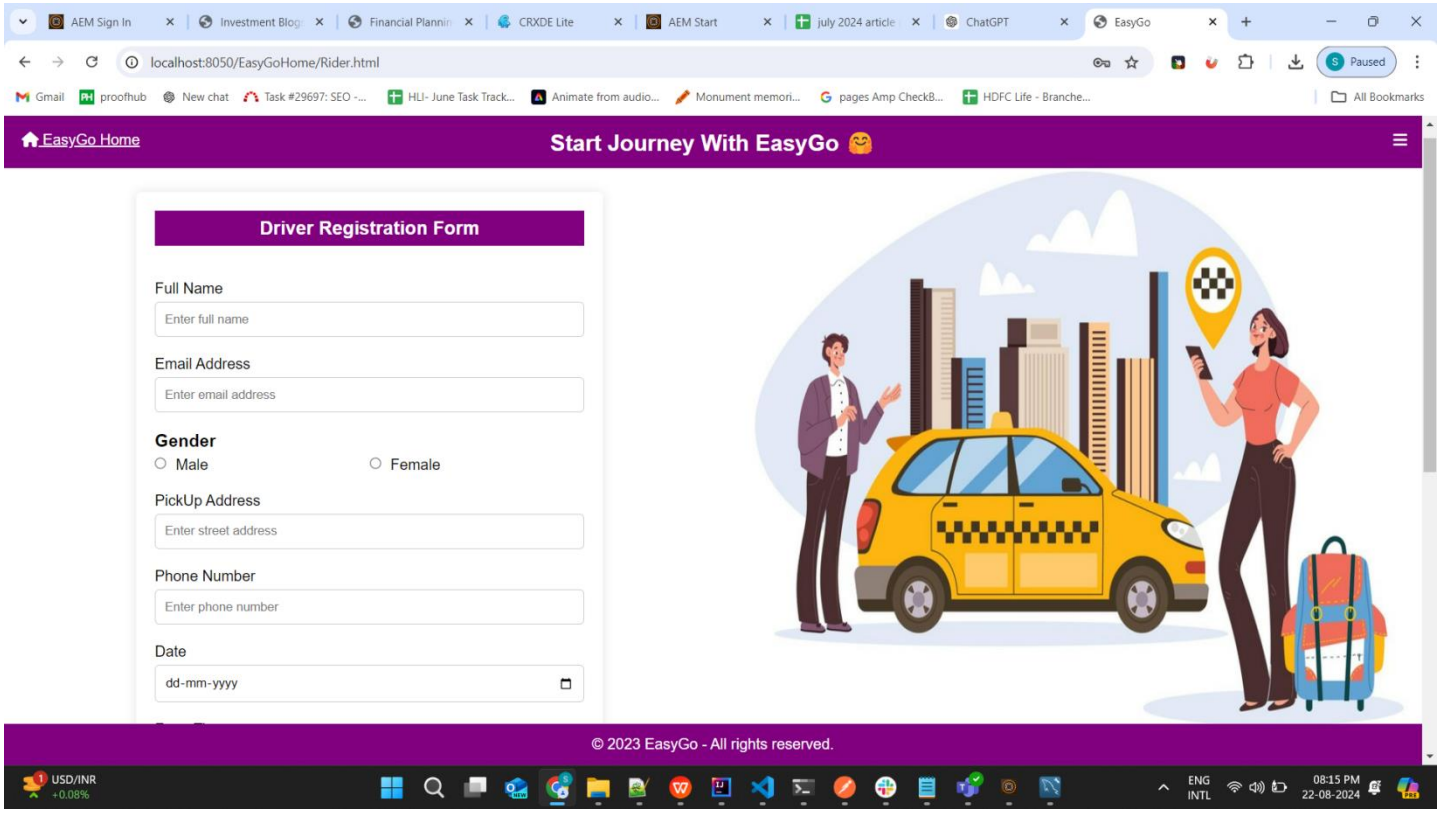

localhost:8050/EasyGoHome/NavigationTab/CategoryPage.html
EasyGo Home
There are two categories of EasyGo
First Category: Driver
Description:
Riders are individuals who provide transportation services to Travellers. EasyGo's Rider category empowers individuals with a vehicle to become drivers and earn income by offering rides to those in need. Riders play a crucial role in creating a connected and accessible transportation network within the EasyGo community.
Driver Registration:
Individuals interested in becoming Riders can easily register on the EasyGo platform, showcasing their availability and vehicle details.
Income Opportunities:
Being a Rider provides an opportunity to earn income by offering rides to Travellers in the vicinity.
Flexible Schedule:
Riders enjoy the flexibility of choosing their working hours and availability, allowing them to balance their personal and professional lives.
© 2023 EasyGo - All rights reserved.

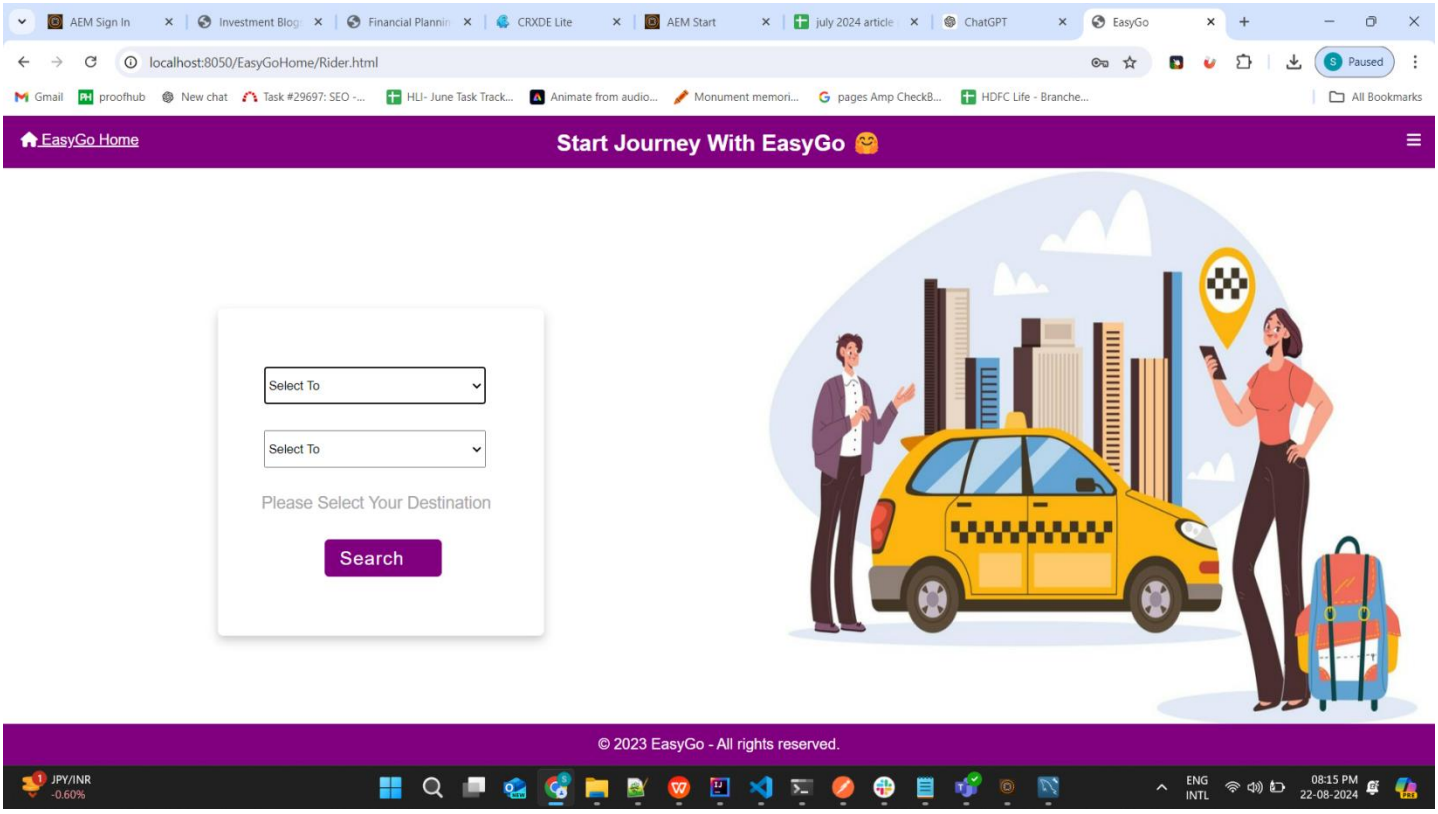

localhost:8050/EasyGoHome/NavigationTab/AboutUsPage.html
EasyGo Home
About EasyGo
Welcome to EasyGo
EasyGo is not just a transportation solution; it's a revolution in the way people move and connect. Established with a vision to make travel convenient, efficient, and enjoyable, EasyGo has been transforming the landscape of transportation since its inception.
Our Mission
At EasyGo, our mission is to provide a seamless and inclusive travel experience. We aim to connect people and places, making transportation easy and accessible for everyone. With a commitment to convenience, efficiency, and environmental sustainability, we strive to redefine the way people travel.
Our Values
EasyGo is built on core values that guide our every decision. We believe in accessibility for all, making sure our platform is user-friendly and caters to diverse needs. Sustainability is at the heart of our operations; we are dedicated to contributing to a greener world through innovative and eco-friendly transportation solutions.
Why Choose EasyGo?
EasyGo stands out in the transportation industry for its commitment to user satisfaction, reliability, and innovation. Our technology-driven approach ensures a hassle-free experience for commuters. Whether it's a daily commute or a long-distance journey, choosing EasyGo means choosing a journey that is not just a means
© 2023 EasyGo - All rights reserved.

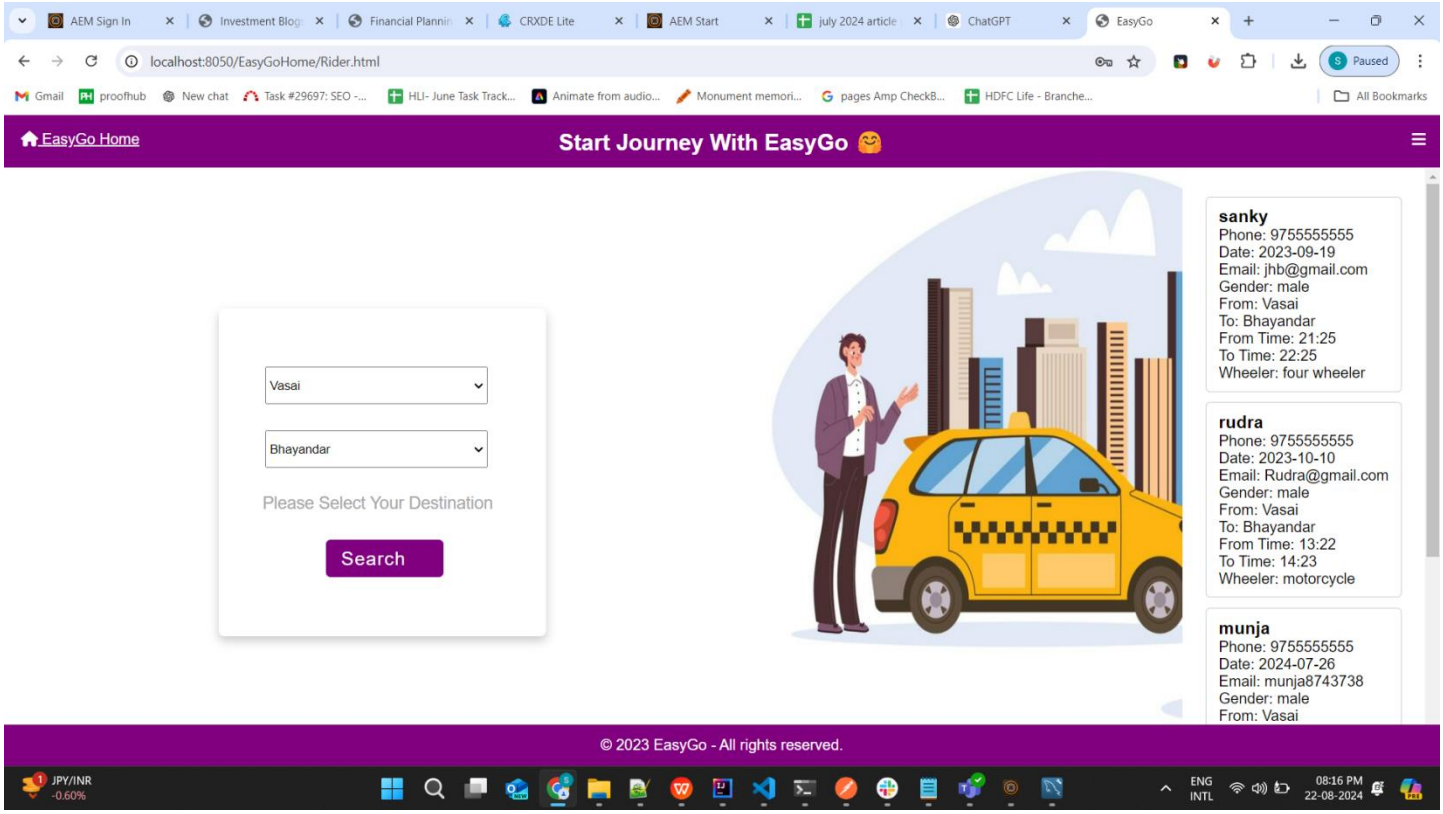

localhost:8050/EasyGoHome/NavigationTab/ContactUsPage.html
EasyGo Home
About EasyGo
Contact Us
If you have any questions or inquiries, feel free to reach out to us:
Email: Sj3308098@gmail.com
Phone: +91 8806-589-844
© 2023 EasyGo - All rights reserved.

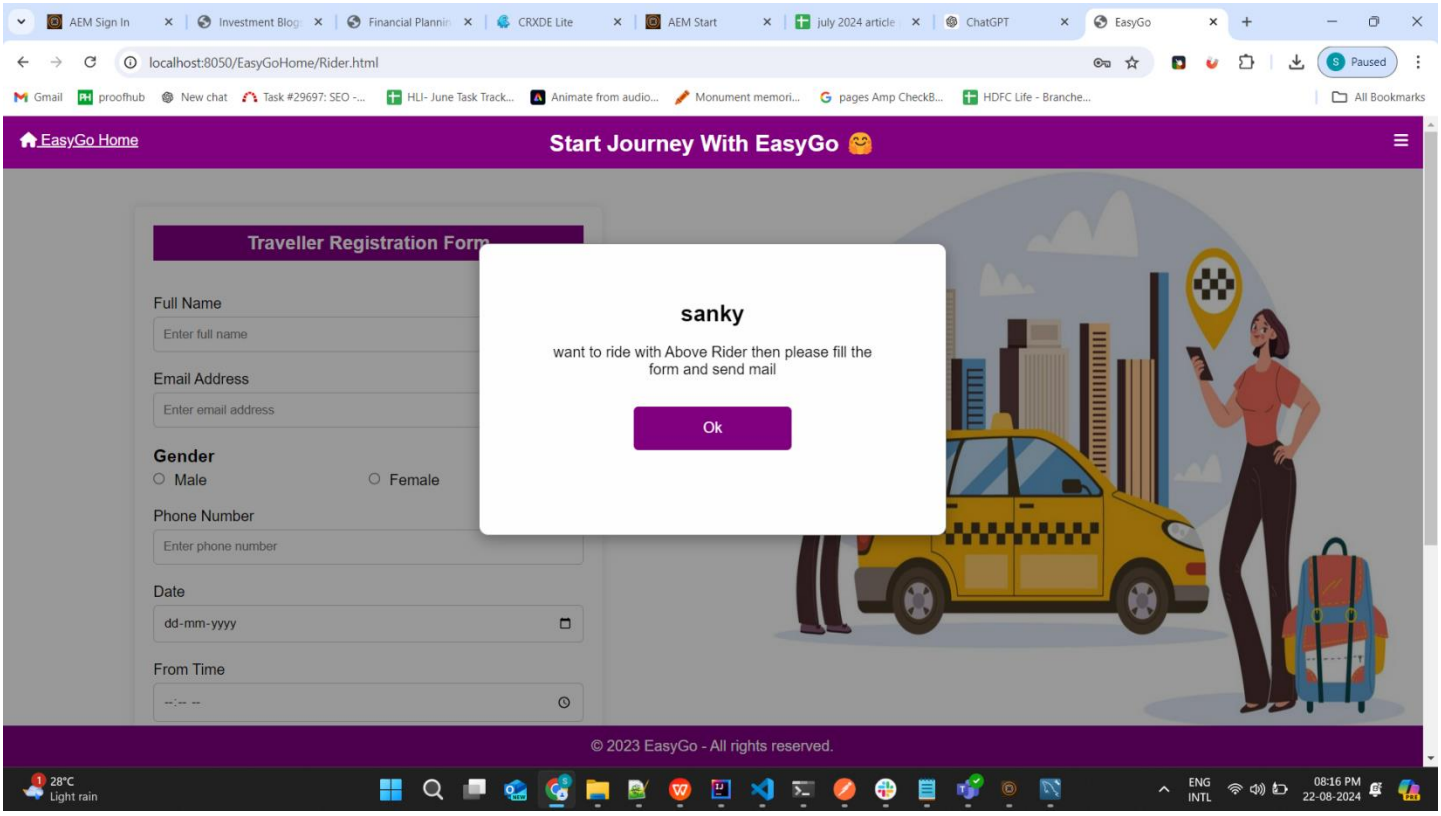

AEM Sign In
Investment Blog
Financial Plannin
CRXDE Lite
AEM Start
july 2024 article
ChatGPT
EasyGo
localhost:8050/EasyGoHome/Rider.html
Paused
Gmail
proofhub
New chat
Task #29697: SEO -...
HLI- June Task Track...
Animate from audio...
Monument memori...
pages Amp CheckB...
HDFC Life - Branche...
All Bookmarks
EasyGo Home
Start Journey With EasyGo 🤗
EasyGoHome Web for EasyGo
Verifying driver details, and sharing trip info. Be punctual and respectful to drivers, maintaining cleanliness. Have payment ready and use emergency features when necessary for a smooth EasyGo ride.
Driver
Traveller
© 2023 EasyGo - All rights reserved.
28°C
Light rain
ENG
INTL
08:14 PM
22-08-2024

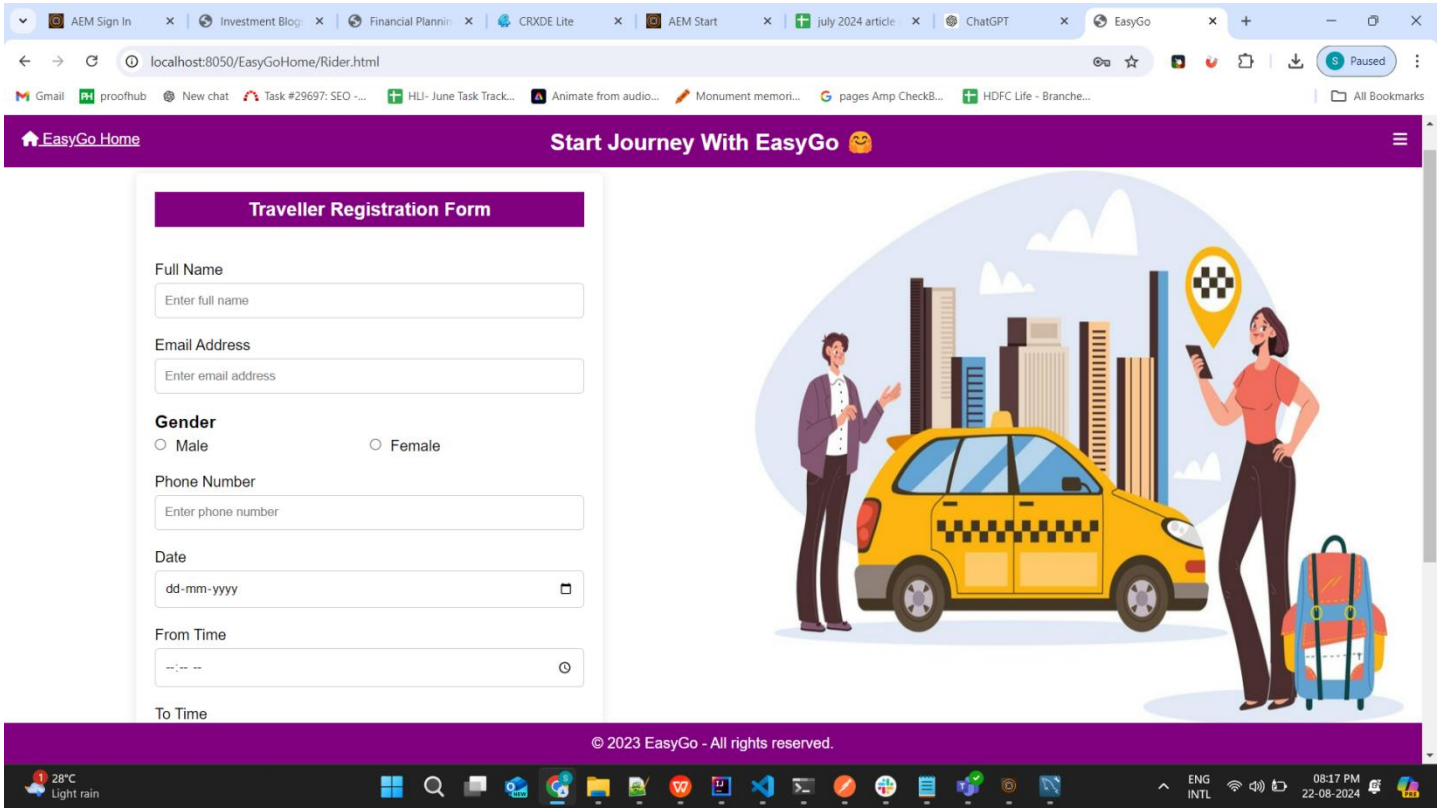

AEM Sign In
Investment Blog
Financial Plannin
CRXDE Lite
AEM Start
july 2024 article
ChatGPT
EasyGo
localhost:8050/EasyGoHome/Rider.html
Paused
Gmail
proofhub
New chat
Task #29697: SEO -...
HLI- June Task Track...
Animate from audio...
Monument memori...
pages Amp CheckB...
HDFC Life - Branche...
All Bookmarks
EasyGo Home
Start Journey With EasyGo 🤗
Driver Registration Form
Full Name
Enter full name
Email Address
Enter email address
Gender
Male
Female
PickUp Address
Enter street address
Phone Number
Enter phone number
Date
dd-mm-yyyy
© 2023 EasyGo - All rights reserved.
USD/INR
+0.08%
ENG
INTL
08:14 PM
22-08-2024

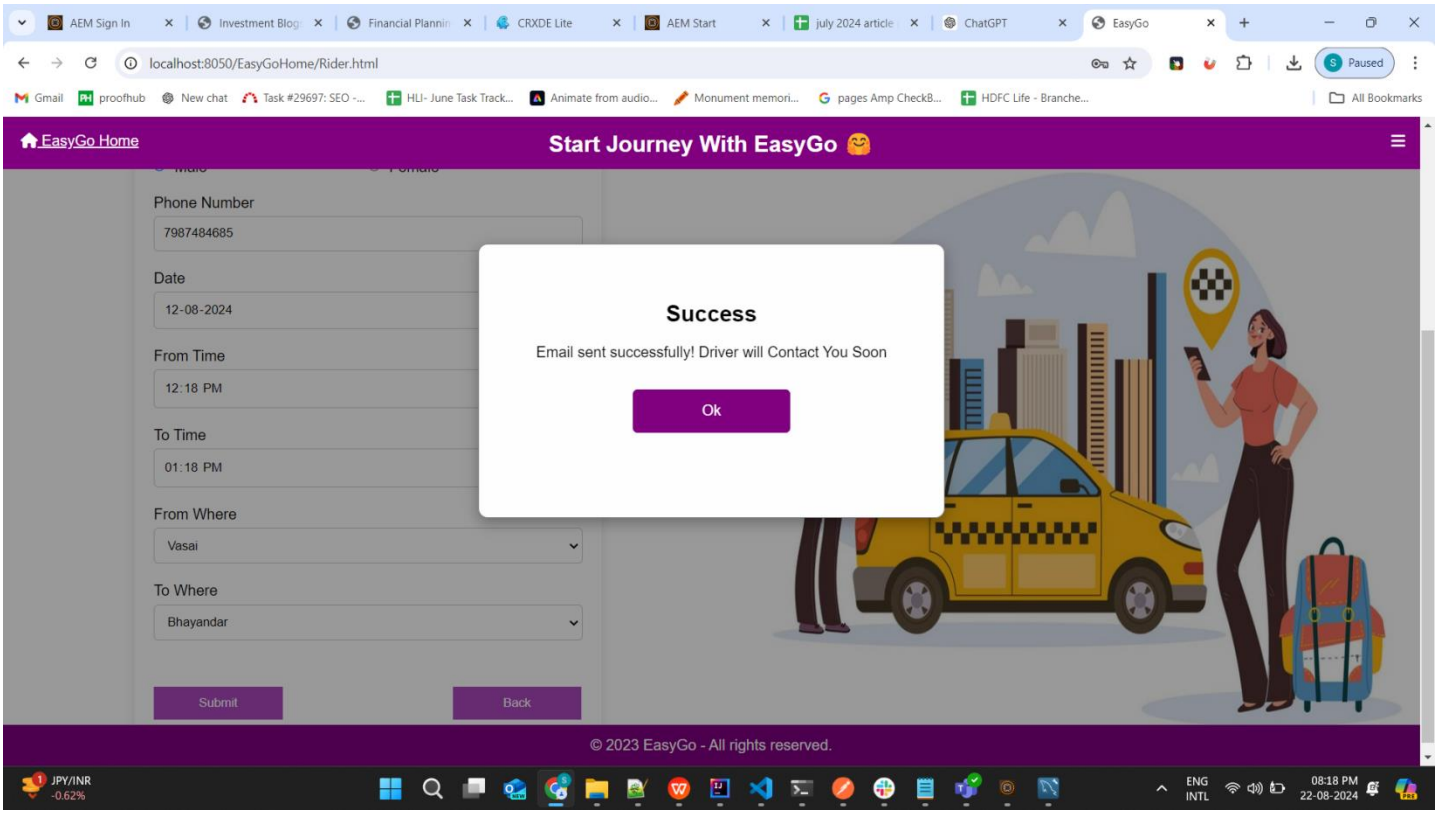

EasyGo Home
Start Journey With EasyGo
Driver Registration Form
Full Name
Enter full name
Email Address
Enter email address
Gender
Male
Female
PickUp Address
Enter street address
Phone Number
Enter phone number
Date
dd-mm-yyyy
© 2023 EasyGo - All rights reserved.


EasyGo Home
Start Journey With EasyGo
Select To
Select To
Please Select Your Destination
Search
© 2023 EasyGo - All rights reserved.


localhost:8050/EasyGoHome/Rider.html
EasyGo Home
Start Journey With EasyGo 🤗
Vasai
Bhayandar
Please Select Your Destination
Search
sanky
Phone: 9755555555
Date: 2023-09-19
Email: jhb@gmail.com
Gender: male
From: Vasai
To: Bhayandar
From Time: 21:25
To Time: 22:25
Wheeler: four wheeler
rudra
Phone: 9755555555
Date: 2023-10-10
Email: Rudra@gmail.com
Gender: male
From: Vasai
To: Bhayandar
From Time: 13:22
To Time: 14:23
Wheeler: motorcycle
munja
Phone: 9755555555
Date: 2024-07-26
Email: munja8743738
Gender: male
From: Vasai
© 2023 EasyGo - All rights reserved.


localhost:8050/EasyGoHome/Rider.html
EasyGo Home
Start Journey With EasyGo 🤗
Traveller Registration Form
Full Name
Enter full name
Email Address
Enter email address
Gender
Male
Female
Phone Number
Enter phone number
Date
dd-mm-yyyy
From Time
sanky
want to ride with Above Rider then please fill the form and send mail
Ok
© 2023 EasyGo - All rights reserved.


localhost:8050/EasyGoHome/Rider.html
EasyGo Home
Start Journey With EasyGo
Traveller Registration Form
Full Name
Enter full name
Email Address
Enter email address
Gender
Male
Female
Phone Number
Enter phone number
Date
dd-mm-yyyy
From Time
To Time
© 2023 EasyGo - All rights reserved.


localhost:8050/EasyGoHome/Rider.html
EasyGo Home
Start Journey With EasyGo
Phone Number
7987484685
Date
12-08-2024
From Time
12:18 PM
To Time
01:18 PM
From Where
Vasai
To Where
Bhayandar
Submit
Back
Success
Email sent successfully! Driver will Contact You Soon
Ok
© 2023 EasyGo - All rights reserved.

Conclusion

In conclusion, the Easy Go Home project presents a comprehensive solution to modern transportation challenges by leveraging technology to facilitate efficient ridesharing experiences. Through the utilization of Java-based technologies, including Servlets, JDBC, and JSP, Easy Go Home offers a user-friendly platform for connecting riders with travelers and optimizing resource utilization. The project addresses technical, economical, and operational feasibility aspects, ensuring a scalable, cost-effective, and user-centric solution.

Moving forward, ongoing maintenance and optimization efforts will be crucial to sustaining the system's functionality, reliability, and performance. By proactively addressing maintenance tasks and continually refining the system based on user feedback, Easy Go Home can maintain its competitive edge in the ridesharing market. Additionally, the emphasis on environmental sustainability and security measures underscores Easy Go's Home commitment to providing a safe, efficient, and environmentally friendly transportation solution. In conclusion, Easy Go Home stands as a testament to the power of technology to transform traditional transportation paradigms and create a more connected, sustainable, and accessible world for all.

BIBLIOGRAPHY

The following YouTube channel and Google website were referred during the analysis and execution phase of the project.

1)"Java: A Beginner's Guide" by Durga software solution.

2)"Java Servlet Programming" by learn with Durgesh.

3)"JavaServer Pages" by learn with Durgesh.

4)"Java Database Best Practices" by Durga software solution.

WEBSITES:

www.google.com
www.microsoft.com
https://www.geeksforgeeks.org/java/
https://www.geeksforgeeks.org/javascript
https://www.javatpoint.com/java-tut